

Face Server Java Wrap API Document 1.9.8-20240704.1-1100

1. 시스템 관련 정보

- 지원 OS: Ubuntu 18.04, Ubuntu 20.04, RedHat 7, RedHat 8

2. 서버 기능 소개

1. 얼굴 검출 (Face Detection)

얼굴 검출은 얼굴 인식 과정의 기초가 되는 작업으로, 이미지를 입력하여 그 안의 얼굴을 모두 찾아냅니다. 검출할 얼굴의 사이즈와 개수 조건을 설정할 수 있습니다. 얼굴 검출만을 지원하는 API도 있으나, 대부분의 API는 얼굴 검출 결과를 포함하기도 합니다.

해당하는 API는 API 상세 챕터의 [POST /faces](#) API의 사용을 확인하시면 됩니다.

2. 얼굴 특징점 추출 (Face Feature Extraction)

특징점 추출은 정해진 사이즈의 얼굴 이미지에서 고유한 **특징점 벡터**를 추출하는 작업입니다. 특징점 벡터는 얼굴마다 고유한 값을 가지며, 두 개의 특징점 벡터를 비교하여 얼굴의 유사도를 파악할 수 있습니다.

해당하는 API는 API 상세 챕터의 [/feature](#) API를 참고하면 됩니다.

특징점 벡터 추출에 사용되는 이미지는 112 x 112 픽셀 크기의 **정렬된 얼굴 이미지**입니다. **얼굴 정렬**은 알체라의 알고리즘을 통해 특징점 추출에 용이한 얼굴 이미지를 얻어내는 작업입니다. 얼굴 정렬 기능이 포함되지 않은 서버 배포판은 API 상세 챕터의 [POST /facefeatures](#) API의 사용을 권장드립니다.

3. 얼굴 정렬 (Face Alignment)

얼굴 정렬은 검출된 얼굴을 바탕으로, 특징점 벡터 추출에 용이하도록 해당 얼굴 이미지를 변형 생성하는 작업입니다. 먼저 이미지에서 얼굴을 검출한 후, 모든 얼굴 이미지가 **코를 기준으로 수평**을 이루도록 정렬됩니다. 또한 생성된 이미지의 크기는 112 x 112로 고정됩니다. 정렬된 얼굴 이미지는 특징점 벡터 추출에 활용 가능합니다. 알체라의 얼굴 정렬 알고리즘으로 생성하지 않은 이미지는 특징점 추출이 부정확할 가능성이 있습니다.

해당하는 API는 API 상세 챕터의 [POST /alignedfaces](#) API의 사용을 확인하시면 됩니다.

4. 얼굴 비교 (Face Compare)

얼굴 비교는 특정한 알고리즘으로 계산된 유사도(similarity confidence) 즉, **두 얼굴이 같은 인물로 판단되는 정도**를 제공합니다. 유사도는 추출된 특징점 벡터를 기반으로 계산됩니다. 유사도가 1에 가까울수록 같은 인물일 확률이 높으며, 0에 가까울수록 다른 인물일 확률이 높습니다.

해당하는 API는 API 상세 챕터의 [POST /compare](#) API의 사용을 확인하시면 됩니다.

3. 요청 데이터 / 응답 코드 목록

3.1. 공통 요청 데이터

API 호출에 대한 정보와 추적성을 제공하기 위해 로그 데이터를 저장하는데 기본으로 저장하는 데이터 (transaction_id, url, ip, status 등) 외에 사용자가 전달하는 값 저장을 목적으로 payload1 ~ payload10까지 request query의 parameter로 전달받아 저장합니다. 모든 API에 공통으로 적용됩니다.

예시)

항목	설명	예시
payload1	거래구분번호	/compare?payload1=213524612
payload2	금융사 코드	/compare?payload2=763452351
payload3	IP	/compare?payload3=192.168.123.132

3.2. 응답 코드 목록

API 호출시 응답 바디에 result_code 및 result_msg 필드가 추가됩니다. 이는 HTTP Status Code와 독립적으로 사용됩니다.

Status Code	return_code	return_msg	Description
200	SUCC-0000	OK.	작업 성공
200	FAIL-0000	face not found.	얼굴 검출 실패
200	FAIL-0001	face size is too small	베스트 샷 추출 실패 - 이미지의 얼굴 크기가 너무 작음
200	FAIL-0002	face size is too large	베스트 샷 추출 실패 - 이미지의 얼굴 크기가 너무 큼
200	FAIL-0003	face is not facing forward.	베스트 샷 추출 실패 - 이미지의 얼굴이 정면을 바라보고 있지 않음
200	FAIL-0004	mask wearing detected on the face of image.	베스트 샷 추출 실패 - 이미지의 얼굴에서 마스크 착용이 감지됨
200	FAIL-0005	face landmark confidence is low.	베스트 샷 추출 실패 - 이미지의 얼굴의 랜드마크 점수가 낮음
200	FAIL-0006	position of the face is not in the center of the image.	베스트 샷 추출 실패 - 이미지에서 얼굴이 가운데에 위치하고 있지 않음
200	FAIL-0007	face detection quality is low.	베스트 샷 추출 실패 - 이미지에서 얼굴 적합성 점수가 낮음
200	FAIL-0008	face is facing downward.	베스트 샷 추출 실패 - 이미지에서 얼굴이 아래를 바라보고 있음
200	FAIL-0009	face is facing upward.	베스트 샷 추출 실패 - 이미지에서 얼굴이 위를 바라보고 있음

Status Code	return_code	return_msg	Description
200	FAIL-0010	face is tilted.	베스트 샷 추출 실패 - 이미지에서 얼굴이 기울어져 있음
200	FAIL-0011	face quality for checking liveness is low.	베스트 샷 추출 실패 - 이미지에서 얼굴의 적합성 점수가 위조판별을 위한 기준보다 낮음
200	FAIL-A000	image 'a' face not found.	얼굴 비교 실패 - a 이미지에서 얼굴 검출 실패
200	FAIL-B000	image 'b' face not found.	얼굴 비교 실패 - b 이미지에서 얼굴 검출 실패
200	FAIL-A001	image 'a' face size is too small	얼굴 비교 실패 - a 이미지의 얼굴 크기가 너무 작음
200	FAIL-B001	image 'b' face size is too small	얼굴 비교 실패 - b 이미지의 얼굴 크기가 너무 작음
200	FAIL-A002	image 'a' face size is too large	얼굴 비교 실패 - a 이미지의 얼굴 크기가 너무 큼
200	FAIL-B002	image 'b' face size is too large	얼굴 비교 실패 - b 이미지의 얼굴 크기가 너무 큼
200	FAIL-A003	image 'a' face is not facing forward	얼굴 비교 실패 - a 이미지의 얼굴이 정면을 바라보고 있지 않음
200	FAIL-B003	image 'b' face is not facing forward	얼굴 비교 실패 - b 이미지의 얼굴이 정면을 바라보고 있지 않음
200	FAIL-A004	mask wearing detected on the face of image 'a'	얼굴 비교 실패 - a 이미지의 얼굴에서 마스크 착용이 감지됨
200	FAIL-B004	mask wearing detected on the face of image 'b'	얼굴 비교 실패 - b 이미지의 얼굴에서 마스크 착용이 감지됨
200	FAIL-A005	image 'a' face detection confidence is low.	얼굴 비교 실패 - a 이미지의 얼굴 검출 신뢰도가 낮음
200	FAIL-B005	image 'b' face detection confidence is low.	얼굴 비교 실패 - b 이미지의 얼굴 검출 신뢰도가 낮음
200	FAIL-A006	image 'a' face detection quality confidence is low.	얼굴 비교 실패 - a 이미지의 얼굴 적합성 점수가 낮음
200	FAIL-B006	image 'b' face detection quality confidence is low.	얼굴 비교 실패 - b 이미지의 얼굴 적합성 점수가 낮음
400	ERR-4001	Request body is invalid.	HTTP Request의 Body가 잘못됨
400	ERR-4002	Request URI Query failed.	HTTP Request의 URI 쿼리 실패
400	ERR-4003	Cannot find uploaded image file (Invalid request body)	업로드된 이미지 파일 없음

Status Code	return_code	return_msg	Description
400	ERR-4004	Content of the image is missing, broken, or incorrect	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
400	ERR-4005	Image to extract feature size not equal with 112x112x3	특징점 추출을 위한 이미지의 크기가 112x112x3이 아님
400	ERR-4006	feature 'a' is invalid format	특징점 비교 실패 - a 특징점의 형식이 부적절함
400	ERR-4007	feature 'b' is invalid format	특징점 비교 실패 - b 특징점의 형식이 부적절함
400	ERR-4008	Field 'frame_counts' is required	'frame_counts' 필드가 존재하지 않음
400	ERR-4009	Field 'frame_counts' is not a number	'frame_counts' 필드의 값이 숫자 형식이 아님
400	ERR-4010	Field 'frame_counts' should be gte 4	'frame_counts' 필드의 값은 4 이상이어야만 함
400	ERR-4011	Field 'filename_extension' is required	'filename_extension' 필드가 존재하지 않음
400	ERR-4012	number of frames is different from given field 'frame_counts'	'frame_counts' 필드의 값이 실제 사진의 개수와 다름
400	ERR-4013	filename extension is different from given field 'filename_extension'	'filename_extension' 필드의 확장자와 실제 사진의 파일 확장자가 다름
400	ERR-4014	resizing factor should be gte '112'	정렬된 얼굴 이미지를 획득할 때 사용하는 resizing 값은 '112' 보다 커야함
400	ERR-4015	dataset file does not exist	민감도 정보를 구성하기 위한 dataset 파일이 존재하지 않음
400	ERR-4016	request body is invalid for setting compare level	민감도 설정 중 올바르지 않은 레벨 값을 요청함
400	ERR-4022	FaceSDK - Multiframe liveness: Cannot unzip requested zip file.	인식 엔진 - Multi frame liveness 요청의 zip 압축파일을 해제할 수 없음
400	ERR-4023	FaceSDK - Multiframe liveness: Invalid image filenames in zip.	인식 엔진 - Multi frame liveness 요청 압축 파일 내 이미지 획득할 수 없음
400	ERR-4024	FaceSDK - Multiframe liveness: Invalid image files count	인식 엔진 - Multi frame liveness 요청 압축 파일 내 이미지 갯수가 다름
406	ERR-4025	API is restricted.	해당 API의 사용이 제한됨
406	ERR-4026	API Server has expired.	API 서버의 사용 기한이 만료됨

Status Code	return_code	return_msg	Description
429	ERR-4027	Server is busy, too many requests.	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	ERR-4501	Maximum upload size exceeded (maximum: 20MB)	wrapper server form 전송 데이터 20MB 초과
400	ERR-4502	MultipartException	이미지 파일 업로드 처리 시 에러
400	ERR-4503	Can not decrypt.	암호화 된 정보를 해독하지 못했을 경우.
400	ERR-4504	Can not encrypt.	암호화에 실패했을 경우.
400	ERR-4505	Can not unzip file.	서버에서 전송받은 압축파일은 압축해제 실패 시 에러
400	ERR-4506	Can not extract bytes from resource.	전송받은 이미지 의 byte정보를 추출하지 못했을 경우.
400	ERR-4507	Unsupported file format.	설정 정보에 대한 검증을 통과 하지 못한 파일의 경우 발생하는 에러.
400	ERR-4508	Can not able to use bank config.	금융사의 config 사용이 활성화 되지 않은 경우.
400	ERR-4509	Some request data empty.	요청시 필요한 정보를 를 보내지 않은 경우
400	ERR-4510	encrypted file is invalid.	요청시 전송한 암호화 파일이 정상적이지 않은 경우
500	ERR-5006	FaceSDK - Initialize Error.	얼굴인식엔진 초기화 에러
500	ERR-5007	FaceSDK - Feature extension Initialize Error.	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	ERR-5008	FaceSDK not ready.	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	ERR-5009	FaceSDK - Set max detectable count failed.	인식 엔진 - 최대 얼굴 검출 개수 설정 에러
500	ERR-5010	FaceSDK - Set min/max detectable size failed.	인식 엔진 - 얼굴 검출 최소/최대 사이즈 설정 에러
500	ERR-5011	FaceSDK - Run DetectInSingleImage failed.	인식 엔진 - 얼굴 검출 중 에러
500	ERR-5012	FaceSDK - Run ExtractFeature failed.	인식 엔진 - 얼굴 특징점 추출 중 에러
500	ERR-5013	Face count unmatched between Detect and AlignFace	검출된 얼굴 개수와 정렬된 얼굴 개수가 다름
500	ERR-5014	FaceSDK - Make Aligned face image failed.	인식 엔진 - 정렬된 얼굴 이미지 추출 중 에러

Status Code	return_code	return_msg	Description
500	ERR-5015	FaceSDK - Deinitialize Error.	인식 엔진 초기화 해제 에러
500	ERR-5016	FaceSDK - Face Validation Checking Error.	인식 엔진 - 얼굴 검출시 신뢰도 확인 에러
500	ERR-5018	FaceSDK - Compute feature distance failed.	유사도 계산 중 에러
500	ERR-5019	FaceSDK - Check face liveness error.	인식 엔진 - 얼굴 진위 여부 확인 중 에러
500	ERR-5020	FaceSDK - Check face occlusions error.	인식 엔진 - 얼굴 가려짐 확인 중 에러
500	ERR-5021	FaceSDK - Check if face wearing a mask error.	인식 엔진 - 얼굴 마스크 착용여부 확인 중 에러
500	ERR-5022	FaceSDK - Attribute extension Initialize Error.	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	ERR-5023	FaceSDK - Antispoofing extension Initialize Error.	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	ERR-5024	Dataset information setup Error.	민감도 정보를 구성하는 과정에서의 에러
500	ERR-5025	FaceSDK - Estimate Quality Check Error.	얼굴 퀄리티 확인을 하는 과정에서의 에러
500	ERR-5026	FaceSDK - Compute Landmark Confidence Error.	얼굴 랜드마크 점수를 얻는 과정에서의 에러
500	ERR-5027	FaceSDK - Estimate Quality For Checking Liveness Error.	위변조판별을 위한 얼굴 적합성 점수를 확인하는 과정에서의 에러
500	ERR-5099	Server is temporarily unavailable	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러
500	ERR-5501	Unexpected error occurred from go server.	go server 와 통신중 예상하지 못한 에러가 발생한 경우.
404	ERR-5502	This url does not exist on go server.	request URI 가 go server 에 존재 하지 않는 경우.
500	ERR-5503	can not load model file.	sdk model 이 존재하지 않은 경우.

4. API 상세

API 목록

No	Method	URI	설명
1	GET	/	서버 상태 확인

No	Method	URI	설명
2	GET	/version	서버 및 FaceSDK 버전 확인
3	GET	/engine/status	알체라의 인공지능 엔진의 초기화 상태 확인
4	POST	/faces	얼굴 검출 결과 요청
5	POST	/feature/floats	얼굴 특징점 추출 결과 요청 (Float array)
6	POST	/feature/bytes	얼굴 특징점 추출 결과 요청 (Byte array)
7	POST	/feature/masking	임의 마스크 착용시를 포함한 얼굴 특징점 추출 결과 요청
8	POST	/facefeatures	얼굴 검출 결과 및 특징점 추출 결과 요청 (Float Array)
9	POST	/facefeatures/withimages	얼굴 검출, 특징점 추출, 정렬된 얼굴 이미지 결과 동시 요청
10	POST	/facefeatures/masking	얼굴 검출, 가상 마스크 착용시를 포함한 특징점 추출 결과 동시 요청
11	POST	/alignedfaces	얼굴 검출 결과 및 정렬된 얼굴 이미지 요청
12	POST	/alignedfaces/v2	얼굴 검출 결과 및 정렬된 얼굴 이미지 요청 - 암호화 지원
12	POST	/alignedfaces/metatemplate	얼굴 검출 결과 및 정렬된 얼굴 이미지 요청(jp2 format의 이미지 반환)
13	POST	/croppedfaces	검출된 얼굴 영역에서 특정 비율로 크롭한 얼굴 이미지를 반환
14	GET	/compare/validation-config	얼굴 매칭시에 사용되는 compare validation 수치 값 반환
15	POST	/compare	얼굴의 매칭 결과 요청(전체 크기 이미지)
16	POST	/compare/feature	특징점 a,b의 매칭 결과 요청(특징점은 base64인코딩된 값)
17	POST	/masks	얼굴 검출 결과 및 얼굴별 가려짐 여부 확인 결과 요청
18	GET	/liveness/validation-config	얼굴 진위 여부(Liveness) 확인에 사용되는 liveness validation 수치 값 반환
19	POST	/liveness	얼굴 진위 여부(Liveness) 확인 - Single frame(사진 1장)
20	POST	/liveness/multiframe	얼굴 진위 여부(Liveness) 확인 - Multi frame mode(사진 4장)
21	POST	/liveness/multiframe/v2	얼굴 진위 여부(Liveness) 확인 - Multi frame mode(사진 4장 이상)
22	POST	/liveness/multiframe/v2/unzip	얼굴 진위 여부(Liveness) 확인 - Multi frame mode(사진 4장 이상,)
23	GET	/threshold	전체 threshold 값 조회
24	GET	/threshold/compare	compare threshold 값 조회
25	PUT	/threshold/compare	compare threshold 값 수정
26	GET	/threshold/liveness	liveness threshold 값 조회

No	Method	URI	설명
27	PUT	/threshold/liveness	liveness threshold 값 수정
28	GET	/version-wrap	face server java wrap 서버 버전 확인
29	GET	/model/info	sdk 버전에 따른 최신 model 정보
30	GET	/model/aos	aos 용 전체 모델 다운로드
31	GET	/model/ios	ios 용 전체 모델 다운로드
32	GET	/model/aos/best-shot	aos 용 best-shot 모델 다운로드
33	GET	/model/aos/liveness	aos 용 liveness 모델 다운로드
34	GET	/model/ios/best-shot	ios 용 best-shot 모델 다운로드
35	GET	/model/ios/liveness	ios 용 liveness 모델 다운로드
36	GET	/model/aos/date	aos 용 전체 모델 모델 파일 수정날짜
37	GET	/model/ios/date	ios 용 전체 모델 모델 파일 수정날짜
38	GET	/model/aos/best-shot/date	aos 용 best-shot 파일 수정날짜
39	GET	/model/aos/liveness/date	aos 용 liveness 모델 파일 수정날짜
40	GET	/model/ios/best-shot/date	ios 용 best-shot 모델 파일 수정날짜
41	GET	/model/ios/liveness/date	ios 용 liveness 모델 파일 수정날짜

4.1. GET /

API 개요

서버의 정상 작동 여부를 확인합니다.

Request Query

Request Header

Request Body

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	
500	오류		

Response Header

항목	내용	비고
----	----	----

항목	내용	비고
Content-type	application/json	API 응답 예시 참조

Response Body

항목	타입	설명
return_code	String	결과 응답 코드 (3.2 응답 코드 목록 참조)
return_msg	String	결과 응답 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

```
curl -X GET http://127.0.0.1:{port}/
{
  "return_code": "SUCC-0000",
  "return_msg": "OK."
}
```

4.2. GET /version

API 개요

서버와 서버가 사용하는 FaceSDK의 버전 정보를 확인합니다.

Request Query

Request Header

Request Body

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	
500	오류		

Response Header

항목	내용	비고
Content-type	application/json	API 응답 예시 참조

Response Body

항목	타입	설명
product_name	String	제품명 정보
product_version	String	제품 버전 정보
server_version	String	서버 버전 정보
facesdk_version	String	서버가 사용하는 FaceSDK 버전 정보
return_msg	String	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

```
curl -X GET http://127.0.0.1:{port}/version
{
  "product_name": "ALCHERA FACE TRUST",
  "product_version": "1.0",
  "server_version": "1.5.0",
  "facesdk_version": "1.14.0",
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

4.3. GET /engine/status

API 개요

알체라의 인공지능 엔진의 초기화 상태를 알려줍니다.

Request Query

Request Header

Request Body

Response Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	알체라의 인공지능 엔진 초기화 성공 상태
500	오류	ERR-5006	얼굴인식엔진 초기화 에러

Response Header

항목	내용	비고
----	----	----

항목	내용	비고
Content-type	application/json	API 응답 예시 참조

Response Body

항목	타입	설명
return_code	String	결과 응답 코드 (3.2 응답 코드 목록 참조)
return_msg	String	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

```
curl -X GET http://127.0.0.1:{port}/engine/status
{
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

4.5. POST /faces

API 개요

여러 개의 얼굴이 포함된 이미지를 전달하여, 이미지 내에서 얼굴을 검출한 결과를 얻어냅니다. 얼굴 검출의 최소 및 최대 사이즈, 최대 검출 갯수를 각각 지정할 수 있습니다. 따로 지정하지 않을 경우 이하에 명시된 기본값을 적용해 얼굴을 검출합니다. 또한 입력 이미지에 얼굴이 없을 경우, 결과 얼굴 count가 0인 것이 정상 동작입니다. 실제로 얼굴을 잡지 못했으니 오동작이 아닙니다.

Request Query

항목	설명	예시
minsize	최소 얼굴 검출 사이즈	/faces?minsize=48
maxsize	최대 얼굴 검출 사이즈	/faces?maxsize=150
detectcount	최대 얼굴 검출 갯수	/faces?detectcount=5

- minsize가 명시되지 않은 경우 48을 기본값으로 사용합니다.
- maxsize가 요청 이미지보다 클 경우 이미지의 크기를 maxsize로 설정합니다.
- detectcount가 명시되지 않은 경우 10을 기본값으로 설정합니다.

Request Header

항목	내용	비고
----	----	----

항목	내용	비고
Content-type	image/jpeg	MIME 타입이 명시되지 않은 경우 image/jpeg를 기본값으로 사용함

Request Body

항목	타입	필수여부	설명
-	JPEG 파일 데이터	Y	얼굴을 검출할 이미지

Response Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
400	오류	ERR-4002	HTTP Request의 URI 쿼리 실패
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	오류	ERR-4504	암호화에 실패했을 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5009	인식 엔진 - 최대 얼굴 검출 개수 설정 에러
500	오류	ERR-5010	인식 엔진 - 얼굴 검출 최소/최대 사이즈 설정 에러
500	오류	ERR-5011	인식 엔진 - 얼굴 검출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
----	----	----

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

얼굴 검출 결과에는 박스 정보 (x, y, width, height), 106점 랜드마크 정보, 얼굴 각도(Yaw, Pitch, Roll) 정보가 포함됩니다.

항목	타입	설명
count	Integer	검출된 얼굴 개수
faces	JSON Object	얼굴 검출 결과 (요청 및 응답 예시 참조)
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

```
curl -X POST http://127.0.0.1:{port}/faces -H 'Content-Type: image/jpeg' -
-data-binary '@test.jpg'
{
  "count": 5,
  "faces": [
    {
      "box": {
        "x": 117.23624,
        "y": 247.9227,
        "width": 202.05891,
        "height": 202.05891
      },
      "landmark": {
        "lm_points": [
          {
            "x": 111.12232,
            "y": 277.64584
          },
          ...
        ]
      },
      "pose": {
        "yaw": -3.6446712,
        "pitch": 14.918827,
        "roll": 0.8090107
      }
    },
    ...
  ],
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

```
    }  
}
```

4.6. POST /feature/floats

API 개요

112 x 112 x 3 크기의 **정렬된 얼굴 이미지**를 전달하여 512개의 실수 값으로 이루어진 얼굴 특징점 벡터를 얻어냅니다. 얼굴 이미지의 크기가 다를 경우 동작하지 않습니다. 또한 알체라 알고리즘으로 정렬한 이미지가 아닐 경우 부정확한 특징점 벡터를 반환할 수 있습니다.

인물마다 고유한 특징점 벡터가 만들어지며, 512 길이의 **Float Array** 형태로 반환됩니다.

Request Query

항목	타입	필수여부	비고
encrypt_mode	Boolean	N	암호화 모드 사용 유무 (디폴트 false)

Request Header

항목	내용	비고
Content-type	image/jpeg	MIME 타입이 명시되지 않은 경우 image/jpeg를 기본값으로 사용함

Request Body

항목	타입	필수여부	설명
-	JPEG 파일 데이터	Y	정렬된 얼굴 이미지 - 112 x 112 x 3 (W x H x C)

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
400	오류	ERR-4005	특징점 추출을 위한 이미지의 크기가 112x112x3이 아님
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우

Status Code	Type	return_code	Description
400	오류	ERR-4503	암호화 된 정보를 해독하지 못했을 경우.
400	오류	ERR-4504	암호화에 실패했을 경우.
400	오류	ERR-4506	전송받은 이미지 의 byte정보를 추출하지 못했을 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5012	인식 엔진 - 얼굴 특징점 추출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

특징점 추출 결과가 512 길이의 Float Array 형태로 반환됩니다.

항목	타입	설명
feature_vector	Float Array	특징점 추출 결과 (Float Array, 요청 및 응답 예시 참조)
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

- SUCC-0000 (암호화 모드)

```
curl -X POST http://127.0.0.1:{port}/feature/floats?encrypt_mode=true -H
'Content-Type: application/octet-stream' --data-binary '@feature_test.jpg'
{
  "feature_vector": [
    0.091061234,
    -0.009058907,
    -0.05616828,
    0.041060984,
```

```
        -0.0218575,
        ...
    ],
    "return_msg": {
        "return_code": "SUCC-0000",
        "return_msg": "OK."
    }
}
```

- SUCC-0000

```
curl -X POST http://127.0.0.1:{port}/feature/floats -H 'Content-Type: image/jpeg' --data-binary '@feature_test.jpg'
{
    "feature_vector": [
        0.091061234,
        -0.009058907,
        -0.05616828,
        0.041060984,
        -0.0218575,
        ...
    ],
    "return_msg": {
        "return_code": "SUCC-0000",
        "return_msg": "OK."
    }
}
```

4.7. POST /feature/bytes

API 개요

112 x 112 x 3 크기의 **정렬된 얼굴 이미지**를 전달하여 512개의 실수 값으로 이루어진 얼굴 특징점 벡터를 얻어냅니다. 얼굴 이미지의 크기가 다를 경우 동작하지 않습니다. 또한 알체라 알고리즘으로 정렬한 이미지가 아닐 경우 부정확한 특징점 벡터를 반환할 수 있습니다.

인물마다 고유한 특징점 벡터가 만들어지며, 2048 길이의 **Byte Array** 형태로 반환됩니다.

Request Query

항목	타입	필수여부	비고
encrypt_mode	Boolean	N	암호화 모드 사용 유무 (디폴트 false)

Request Header

항목	내용	비고
----	----	----

항목	내용	비고
Content-type	image/jpeg	MIME 타입이 명시되지 않은 경우 image/jpeg를 기본값으로 사용함

Request Body

항목	타입	필수여부	설명
-	JPEG 파일 데이터	Y	정렬된 얼굴 이미지 - 112 x 112 x 3 (W x H x C)

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
400	오류	ERR-4005	특징점 추출을 위한 이미지의 크기가 112x112x3이 아님
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	오류	ERR-4503	암호화 된 정보를 해독하지 못했을 경우.
400	오류	ERR-4504	암호화에 실패했을 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5012	인식 엔진 - 얼굴 특징점 추출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
----	----	----

항목	내용	비고
Content-type	application/octet-stream application/json	정상 동작시 : 실행 결과가 전달됨 오류 발생시 : 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

정상 동작시 특징점 추출 결과가 2048 길이의 Byte Array 형태로 반환됩니다.

항목	타입	설명
-	Byte Array	특징점 추출 결과 (Byte Array, 요청 및 응답 예시 참조)

오류 발생시 return_msg JSON 객체만 반환됩니다. 응답 객체의 구조 자체는 [POST /feature/floats](#) API와 동일하지만, feature_vector 필드는 항상 null로 들어옵니다.

항목	타입	설명
feature_vector	JSON Object	항상 null (무시 가능)
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

- (암호화 모드)

```
curl -X POST http://127.0.0.1:{port}/feature/bytes?encrypt_mode=true -H
'Content-Type: application/octet-stream' --data-binary '@feature_test.jpg'
--output feature_result.bin
```

```
curl -X POST http://127.0.0.1:{port}/feature/bytes -H 'Content-Type: image/jpeg' --data-binary '@feature_test.jpg' --output feature_result.bin
```

- 동작 성공시

Byte String0이 응답으로 전달됩니다. 다음의 문자열은 Byte String임을 표현하기 위한 것으로, 정상입니다.

```
E@R;?ep<[G... (Byte string)
```

- 오류 발생시

```
{
  "feature_vector": null,
  "return_msg": {
```

```
    "return_code": "ERR-4003",
    "return_msg": "Cannot find uploaded image file (Invalid request body)"
  }
}
```

4.8. POST /feature/masking

API 개요

112 x 112 x 3 크기의 **정렬된 얼굴 이미지**를 전달하여 얼굴 특징점 벡터 **2개**를 얻어냅니다. 하나는 원본 이미지에서 추출, 다른 하나는 특정 알고리즘으로 임의의 마스크를 씌운 이미지에서 추출합니다. 동일 인물이 마스크를 쓰고 두 번 인식할 필요 없이, 원본 사진으로 마스크를 착용한 상황의 특징점까지 한 번에 얻어내기 위함입니다. 얼굴 이미지의 크기가 다를 경우 동작하지 않습니다. 또한 알체라 알고리즘으로 정렬한 이미지가 아닐 경우 부정확한 특징점 벡터를 반환할 수 있습니다.

특징점 벡터는 각각 **Base64 인코딩된 2048 길이의 Byte Array** 형태로 반환됩니다.

Request Query

Request Header

항목	내용	비고
Content-type	image/jpeg	MIME 타입이 명시되지 않은 경우 image/jpeg를 기본값으로 사용함

Request Body

항목	타입	필수여부	설명
-	JPEG 파일 데이터	Y	정렬된 얼굴 이미지 - 112 x 112 x 3 (W x H x C)

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
200	실패	FAIL-0000	얼굴 검출 실패
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
400	오류	ERR-4005	특징점 추출을 위한 이미지의 크기가 112x112x3이 아님
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우

Status Code	Type	return_code	Description
400	오류	ERR-4504	암호화에 실패했을 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5009	인식 엔진 - 최대 얼굴 검출 개수 설정 에러
500	오류	ERR-5010	인식 엔진 - 얼굴 검출 최소/최대 사이즈 설정 에러
500	오류	ERR-5011	인식 엔진 - 얼굴 검출 중 에러
500	오류	ERR-5012	인식 엔진 - 얼굴 특징점 추출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

특징점 추출 결과가 **Base64** 인코딩된 **2048** 길이 **Byte Array** 형태로 반환됩니다.

항목	타입	설명
normal_feature	Bytes	원본 이미지 특징점 추출 결과 (요청 및 응답 예시 참조)
masked_feature	Bytes	임의 마스킹된 이미지 특징점 추출 결과 (요청 및 응답 예시 참조)
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

```
curl -X POST http://127.0.0.1:{port}/feature/masking -H 'Content-Type: image/jpeg' --data-binary '@feature_test.jpg'
{
  "normal_feature": "dzq2PWKjKLwK...",
  "masked_feature": "jgP2PdT3k7yc...",
  "return_msg": {
```

```
        "return_code": "SUCC-0000",
        "return_msg": "OK."
    }
}
```

4.9. POST /facefeatures

API 개요

여러 개의 얼굴이 포함된 이미지를 전달하여, 이미지 내의 얼굴 검출 결과와 각각의 특징점 추출 결과를 동시에 얻어냅니다. 얼굴 검출의 최소 및 최대 사이즈, 최대 검출 갯수를 각각 지정할 수 있습니다. 따로 지정하지 않을 경우 이하에 명시된 기본값을 적용해 얼굴을 검출합니다. 특징점 추출 결과는 [POST /feature/floats](#) API와 동일하게 각각 512 길이의 Float Array로 반환됩니다.

Request Query

항목	설명	예시
minsize	최소 얼굴 검출 사이즈	/facefeatures?minsize=48
maxsize	최대 얼굴 검출 사이즈	/facefeatures?maxsize=150
detectcount	최대 얼굴 검출 갯수	/facefeatures?detectcount=5
encrypt_mode	암호화 모드 사용 유무 (디폴트 false)	/facefeatures?encrypt_mode=true
feature_format	특징점 형식 base64 반환 유무	/facefeatures?feature_format=base64

- minsize가 명시되지 않은 경우 48을 기본값으로 사용합니다.
- maxsize가 요청 이미지보다 클 경우 이미지의 크기를 maxsize로 설정합니다.
- detectcount가 명시되지 않은 경우 10을 기본값으로 설정합니다.

feature_format을 별도로 명시하지 않은 경우 **feature_vector**(float array 타입)를 반환하고 feature_format=base64로 지정한다면 **feature_base64**(string 타입)를 반환합니다. (자세한 사항은 아래의 **요청 및 응답 예시**를 참고해주세요.)

Request Header

항목	내용	비고
Content-type	image/jpeg	MIME 타입이 명시되지 않은 경우 image/jpeg를 기본값으로 사용함

Request Body

항목	타입	필수여부	설명
-	JPEG 파일 데이터	Y	얼굴 검출 및 특징점 추출을 수행할 이미지

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
400	오류	ERR-4002	HTTP Request의 URI 쿼리 실패
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	오류	ERR-4503	암호화 된 정보를 해독하지 못했을 경우.
400	오류	ERR-4504	암호화에 실패했을 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5009	인식 엔진 - 최대 얼굴 검출 개수 설정 에러
500	오류	ERR-5010	인식 엔진 - 얼굴 검출 최소/최대 사이즈 설정 에러
500	오류	ERR-5011	인식 엔진 - 얼굴 검출 중 에러
500	오류	ERR-5012	인식 엔진 - 얼굴 특징점 추출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

얼굴 검출 결과는 박스 정보(x, y, width, height) 만 제공되며, 특징점 추출 결과의 포맷은 512 길이의 Float Array 혹은 base64 인코드 된 String입니다.

항목	타입	설명
----	----	----

항목	타입	설명
count	Integer	검출된 얼굴 개수
faces_with_features	JSON Object	얼굴 검출 및 각각의 특징점 추출 결과 (Float Array, 요청 및 응답 예시 참조)
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

- SUCC-0000 (암호화 모드)

```
curl -X POST http://127.0.0.1:{port}/facefeatures?encrypt_mode=true -H
'Content-Type: application/octet-stream' --data-binary '@test.jpg'
{
  "count": 5,
  "faces_with_features": [
    {
      "box": {
        "x": 117.23624,
        "y": 247.9227,
        "width": 202.05891,
        "height": 202.05891
      },
      "pose": {
        "yaw": -1.452313,
        "pitch": 13.3671255,
        "roll": 0.12734002
      },
      "feature_vector": [
        0.0020271898,
        -0.06834793,
        0.014104067,
        0.048046052,
        ...
      ]
    },
    ...
  ],
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

- SUCC-0000

```
curl -X POST http://127.0.0.1:{port}/facefeatures -H 'Content-Type:
image/jpeg' --data-binary '@test.jpg'
```

```
{
  "count": 5,
  "faces_with_features": [
    {
      "box": {
        "x": 117.23624,
        "y": 247.9227,
        "width": 202.05891,
        "height": 202.05891
      },
      "pose": {
        "yaw": -1.452313,
        "pitch": 13.3671255,
        "roll": 0.12734002
      },
      "feature_vector": [
        0.0020271898,
        -0.06834793,
        0.014104067,
        0.048046052,
        ...
      ]
    },
    ...
  ],
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

- **feature_format=base64** 로 명시한 경우

```
curl -X POST http://127.0.0.1:{port}/facefeatures?feature_format=base64 --
data-binary '@test.jpg'
{
  "count": 5,
  "faces_with_features": [
    {
      "box": {
        "x": 117.23624,
        "y": 247.9227,
        "width": 202.05891,
        "height": 202.05891
      },
      "pose": {
        "yaw": -1.452313,
        "pitch": 13.3671255,
        "roll": 0.12734002
      },
      "feature_base64": "dzq2PWKjKLwK..." # base64 인코딩 특징점 데이터
    },
    ...
  ],
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```



```
    ...
  ],
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

4.10. POST /facefeatures/withimages

API 개요

여러 개의 얼굴이 포함된 이미지를 전달하여, 이미지 내의 얼굴 검출 결과와 각각의 특징점 추출 결과, 정렬된 얼굴 이미지들을 동시에 얻어냅니다. 얼굴 검출의 최소 및 최대 사이즈, 최대 검출 갯수를 각각 지정할 수 있습니다. 따로 지정하지 않을 경우 이하에 명시된 기본값을 적용해 얼굴을 검출합니다. 특징점 추출 결과는 [POST /feature/floats](#) API와 동일하게 각각 512 길이의 Float Array 형태로 반환됩니다. 정렬된 얼굴 이미지는 **112 x 112 x 3 사이즈의 JPG 포맷 RGB 영상**으로, **base64 인코딩**되어 반환됩니다.

Request Query

항목	설명	예시
minsize	최소 얼굴 검출 사이즈	/facefeatures/withimages?minsize=48
maxsize	최대 얼굴 검출 사이즈	/facefeatures/withimages?maxsize=150
detectcount	최대 얼굴 검출 갯수	/facefeatures/withimages?detectcount=5
feature_format	특징점 형식 base64 반환 유무	/facefeatures?feature_format=base64

- minsize가 명시되지 않은 경우 48을 기본값으로 사용합니다.
- maxsize가 요청 이미지보다 클 경우 이미지의 크기를 maxsize로 설정합니다.
- detectcount가 명시되지 않은 경우 10을 기본값으로 설정합니다.

feature_format을 별도로 명시하지 않은 경우 **feature_vector**(float array 타입)를 반환하고 feature_format=base64로 지정한다면 **feature_base64**(string 타입)를 반환합니다. (자세한 사항은 아래의 **요청 및 응답 예시**를 참고해주세요.)

Request Header

항목	내용	비고
Content-type	image/jpeg	MIME 타입이 명시되지 않은 경우 image/jpeg를 기본값으로 사용함

Request Body

항목	타입	필수여부	설명
-	JPEG 파일 데이터	Y	얼굴 검출 및 특징점 추출을 수행할 이미지

Response Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
400	오류	ERR-4002	HTTP Request의 URI 쿼리 실패
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	오류	ERR-4504	암호화에 실패했을 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5009	인식 엔진 - 최대 얼굴 검출 개수 설정 에러
500	오류	ERR-5010	인식 엔진 - 얼굴 검출 최소/최대 사이즈 설정 에러
500	오류	ERR-5011	인식 엔진 - 얼굴 검출 중 에러
500	오류	ERR-5012	인식 엔진 - 얼굴 특징점 추출 중 에러
500	오류	ERR-5014	인식 엔진 - 정렬된 얼굴 이미지 추출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

얼굴 검출 결과는 박스 정보(x, y, width, height) 만 제공되며, 특징점 추출 결과의 포맷은 [POST /feature/floats](#) API와 동일하게 512 길이의 Float Array입니다. base64로 인코딩된 얼굴 이미지 데이터가 함께 전달됩니다.

항목	타입	설명
count	Integer	검출된 얼굴 개수
facefeatures_with_images	JSON Object	얼굴 검출 및 각각의 특징점, 정렬된 얼굴 이미지 결과 (요청 및 응답 예시 참조)
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

```
curl -X POST http://127.0.0.1:{port}/facefeatures/withimages -H 'Content-Type: image/jpeg' --data-binary '@test.jpg'
```

```
{
  "count": 5,
  "facefeatures_with_images": [
    {
      "box": {
        "x": 117.23624,
        "y": 247.9227,
        "width": 202.05891,
        "height": 202.05891
      },
      "pose": {
        "yaw": -1.452313,
        "pitch": 13.3671255,
        "roll": 0.12734002
      },
      "feature_vector": [
        0.0020271898,
        -0.06834793,
        0.014104067,
        0.048046052,
        ...
      ],
      "image_data": "SVlgTVphS1dfTVxNr..." # base64 인코딩 얼굴 이미지
    },
    ...
  ],
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

- **feature_format=base64** 로 명시한 경우

```
curl -X POST http://127.0.0.1:{port}/facefeatures/withimages?
feature_format=base64 --data-binary '@test.jpg'
```

```

{
  "count": 5,
  "facefeatures_with_images": [
    {
      "box": {
        "x": 117.23624,
        "y": 247.9227,
        "width": 202.05891,
        "height": 202.05891
      },
      "pose": {
        "yaw": -1.452313,
        "pitch": 13.3671255,
        "roll": 0.12734002
      },
      "feature_base64": "dzq2PWKjKLwK...", # base64 인코딩 특징점 데이터
      "image_data": "SVlgTVphS1dfTVxNr..." # base64 인코딩 얼굴 이미지
    },
    ...
  ],
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}

```

4.11. POST /facefeatures/masking

API 개요

여러 개의 얼굴이 포함된 이미지를 전달하여, 다음의 내용들을 동시에 얻어냅니다.

- 이미지 내의 얼굴 검출 결과
- 각각의 일반 특징점 추출 결과
- 각각의 가상 마스크 적용 특징점 추출 결과

가상 마스크 적용 및 특징점 추출 과정은 [POST /feature/masking](#) API와 동일합니다. 얼굴 검출의 최소 및 최대 사이즈, 최대 검출 갯수를 각각 지정할 수 있습니다. 따로 지정하지 않을 경우 이하에 명시된 기본값을 적용해 얼굴을 검출합니다. 특징점 추출 결과는 [POST /feature/floats](#) API와 동일하게 각각 512 길이의 Float Array로 반환됩니다.

Request Query

항목	설명	예시
minsize	최소 얼굴 검출 사이즈	/facefeatures/masking?minsize=48
maxsize	최대 얼굴 검출 사이즈	/facefeatures/masking?maxsize=150
detectcount	최대 얼굴 검출 갯수	/facefeatures/masking?detectcount=5

항목	설명	예시
feature_format	특징점 형식 base64 반환 유무	/facefeatures?feature_format=base64
- minsize가 명시되지 않은 경우 48을 기본값으로 사용합니다. - maxsize가 요청 이미지보다 클 경우 이미지의 크기를 maxsize로 설정합니다. - detectcount가 명시되지 않은 경우 10을 기본값으로 설정합니다. feature_format을 별도로 명시하지 않은 경우 normal_feature, masked_feature(float array 타입)를 반환하고 feature_format=base64로 지정한다면 normal_feature_base64, masked_feature_base64(string 타입)를 반환합니다. (자세한 사항은 아래의 요청 및 응답 예시 를 참고해주세요.)		

Request Header

항목	내용	비고
Content-type	image/jpeg	MIME 타입이 명시되지 않은 경우 image/jpeg를 기본값으로 사용함

Request Body

항목	타입	필수여부	설명
-	JPEG 파일 데이터	Y	얼굴 검출 및 특징점 추출을 수행할 이미지

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
400	오류	ERR-4002	HTTP Request의 URI 쿼리 실패
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	오류	ERR-4504	암호화에 실패했을 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5009	인식 엔진 - 최대 얼굴 검출 개수 설정 에러
500	오류	ERR-5010	인식 엔진 - 얼굴 검출 최소/최대 사이즈 설정 에러

Status Code	Type	return_code	Description
500	오류	ERR-5011	인식 엔진 - 얼굴 검출 중 에러
500	오류	ERR-5012	인식 엔진 - 얼굴 특징점 추출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

얼굴 검출 결과는 박스 정보(x, y, width, height) 만 제공되며, 특징점 추출 결과의 포맷은 [POST /feature/floats](#) API와 동일하게 512 길이의 Float Array입니다.

항목	타입	설명
count	Integer	검출된 얼굴 개수
faces_with_masked_features	JSON Object	얼굴 검출 및 각각의 특징점 추출 결과 (Float Array, 요청 및 응답 예시 참조)
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

```
curl -X POST http://127.0.0.1:{port}/facefeatures/masking -H 'Content-Type: image/jpeg' --data-binary '@test.jpg'
{
  "count": 5,
  "faces_with_masked_features": [
    {
      "box": {
        "x": 117.23624,
        "y": 247.9227,
        "width": 202.05891,
        "height": 202.05891
      },
      "pose": {
        "yaw": -1.452313,
```

```

        "pitch": 13.3671255,
        "roll": 0.12734002
    },
    "normal_feature": [
        0.0020271898,
        -0.06834793,
        0.014104067,
        ...
    ],
    "masked_feature": [
        0.0020271898,
        -0.06834793,
        0.014104067,
        ...
    ]
},
...
],
"return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
}
}

```

- **feature_format=base64** 로 명시한 경우

```

curl -X POST http://127.0.0.1:{port}/facefeatures/masking?
feature_format=base64 --data-binary '@test.jpg'
{

```

```

    "count": 5,
    "faces_with_masked_features": [
        {
            "box": {
                "x": 117.23624,
                "y": 247.9227,
                "width": 202.05891,
                "height": 202.05891
            },
            "pose": {
                "yaw": -1.452313,
                "pitch": 13.3671255,
                "roll": 0.12734002
            },
            "normal_feature_base64": "dzq2PWKjKLwK...", # base64 인코딩 특징점
            "masked_feature_base64": "jgP2PdT3k7yc...", # 가상의 마스크 착용한
            # base64 인코딩 특징점 데이터
        },
        ...
    ],
    "return_msg": {
        "return_code": "SUCC-0000",

```

```
        "return_msg": "OK."
    }
}
```

4.12. POST /alignedfaces

API 개요

여러 개의 얼굴이 포함된 이미지를 전달하여, 이미지 내의 얼굴 검출 결과와 각각의 정렬된 얼굴 이미지를 동시에 얻어냅니다. 얼굴 검출의 최소 및 최대 사이즈, 최대 검출 갯수를 각각 지정할 수 있습니다. 따로 지정하지 않을 경우 이하에 명시된 기본값을 적용해 얼굴을 검출합니다. 또한 `resize_width` 혹은 `resize_height` 값을 지정해서 얼굴 이미지의 크기를 조절할 수 있습니다. `resize_width`, `resize_height` 값을 지정하지 않는다면 기본값인 **112 x 112 x 3** 사이즈의 **JPG 포맷 RGB 영상**의 정렬된 얼굴 이미지가 **base64 인코딩**되어 반환됩니다.

Request Query

항목	설명	예시
minsize	최소 얼굴 검출 사이즈	/alignedfaces?minsize=48
maxsize	최대 얼굴 검출 사이즈	/alignedfaces?maxsize=150
detectcount	최대 얼굴 검출 갯수	/alignedfaces?detectcount=5
resize_width	사진의 너비 사이즈	alignedfaces?resize_width=150
resize_height	사진의 높이 사이즈	alignedfaces?resize_height=200

- minsize가 명시되지 않은 경우 24을 기본값으로 사용합니다.
- maxsize가 요청 이미지보다 클 경우 이미지의 크기를 maxsize로 설정합니다.
- detectcount가 명시되지 않은 경우 10을 기본값으로 설정합니다.
- resize_width를 명시하지 않을 경우 너비 사이즈는 기본값인 112 사이즈로 조절됩니다.
- resize_height를 명시하지 않을 경우 높이 사이즈는 기본값인 112 사이즈로 조절됩니다.

Request Header

항목	내용	비고
Content-type	image/jpeg	MIME 타입이 명시되지 않은 경우 image/jpeg를 기본값으로 사용함

Request Body

항목	타입	필수여부	설명
-	JPEG 파일 데이터	Y	얼굴 검출 및 특징점 추출을 수행할 이미지

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
400	오류	ERR-4002	HTTP Request의 URI 쿼리 실패
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
400	오류	ERR-4014	정렬된 얼굴 이미지를 획득할 때 사용하는 resizing 값은 '112' 보다 커야 함
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	오류	ERR-4504	암호화에 실패했을 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5009	인식 엔진 - 최대 얼굴 검출 개수 설정 에러
500	오류	ERR-5010	인식 엔진 - 얼굴 검출 최소/최대 사이즈 설정 에러
500	오류	ERR-5011	인식 엔진 - 얼굴 검출 중 에러
500	오류	ERR-5013	인식 엔진 - 정렬된 얼굴 이미지 추출 중 에러
500	오류	ERR-5014	인식 엔진 - 정렬된 얼굴 이미지 추출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

얼굴 검출 결과는 박스 정보(x, y, width, height) 만 제공되며, 정렬된 얼굴 이미지의 사이즈 정보와 base64로 인코딩된 이미지 데이터가 함께 전달됩니다.

항목	타입	설명
count	Integer	검출된 얼굴 개수
aligned_faces	JSON Array	얼굴 검출 및 각각의 정렬된 이미지 결과 (요청 및 응답 예시 참조)
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

```
curl -X POST http://127.0.0.1:{port}/alignedfaces -H 'Content-Type: image/jpeg' --data-binary '@test.jpg'
{
  "count": 5,
  "aligned_faces": [
    {
      "box": {
        "x": 117.23624,
        "y": 247.9227,
        "width": 202.05891,
        "height": 202.05891
      },
      "image_height": 112,
      "image_width": 112,
      "image_channel": 3,
      "image_data": "SVlgTVphS1dfTVxNr..." # base64 인코딩 얼굴 이미지
    },
    ...
  ],
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

```
curl -X POST 'http://127.0.0.1:{port}/alignedfaces?resize_height=150&resize_width=150' -H 'Content-Type: image/jpeg' --data-binary '@test.jpg'
{
  "count": 1,
  "aligned_faces": [
    {
      "box": {
        "x": 117.23624,
        "y": 247.9227,
```

```

        "width": 202.05891,
        "height": 202.05891
    },
    "image_height": 150,
    "image_width": 150,
    "image_channel": 3,
    "image_data": "SVlgTVphS1dfTVxNr..." # base64 인코딩 얼굴 이미지
데이터
    }
  ],
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}

```

4.13. POST /alignedfaces/v2

API 개요

여러 개의 얼굴이 포함된 이미지 또는 암호화된 이미지를 전달하여, 이미지 내의 얼굴 검출 결과와 각각의 정렬된 얼굴 이미지를 동시에 얻어냅니다. 얼굴 검출의 최소 및 최대 사이즈, 최대 검출 갯수를 각각 지정할 수 있습니다. 따로 지정하지 않을 경우 이하에 명시된 기본값을 적용해 얼굴을 검출합니다. 또한 `resize_width` 혹은 `resize_height` 값을 지정해서 얼굴 이미지의 크기를 조절할 수 있습니다. `resize_width`, `resize_height` 값을 지정하지 않는다면 기본값인 **112 x 112 x 3 사이즈의 JPG 포맷 RGB 영상**의 정렬된 얼굴 이미지가 **base64 인코딩**되어 반환됩니다.

Request Query

항목	설명	예시
minsize	최소 얼굴 검출 사이즈	/alignedfaces?minsize=48
maxsize	최대 얼굴 검출 사이즈	/alignedfaces?maxsize=150
detectcount	최대 얼굴 검출 갯수	/alignedfaces?detectcount=5
resize_width	사진의 너비 사이즈	alignedfaces?resize_width=150
resize_height	사진의 높이 사이즈	alignedfaces?resize_height=200

- minsize가 명시되지 않은 경우 24을 기본값으로 사용합니다.
- maxsize가 요청 이미지보다 클 경우 이미지의 크기를 maxsize로 설정합니다.
- detectcount가 명시되지 않은 경우 10을 기본값으로 설정합니다.
- resize_width를 명시하지 않을 경우 너비 사이즈는 기본값인 112 사이즈로 조절됩니다.
- resize_height를 명시하지 않을 경우 높이 사이즈는 기본값인 112 사이즈로 조절됩니다.

Request Header

항목	내용	비고
Content-type	multipart/form-data	

Request Body

항목	타입	필수 여부	설명
image	JPEG 파일 데이터	N	얼굴 검출 및 특징점 추출을 수행할 이미지 (encrypt_mode 미 사용 시 필수)
encrypt_mode	Boolean	N	암호화 모드 사용 유무 (디폴트 false)
encrypted_image	String 파일 데이터	N	얼굴 검출 및 특징점 추출을 수행할 이미지 암호화 된 파일 (encrypt_mode 사용 시 필수)

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
400	오류	ERR-4002	HTTP Request의 URI 쿼리 실패
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
400	오류	ERR-4014	정렬된 얼굴 이미지를 획득할 때 사용하는 resizing 값은 '112' 보다 커야 함
400	오류	ERR-4501	wrapper server form 전송 데이터 20MB 초과
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	오류	ERR-4503	암호화 된 정보를 해독하지 못했을 경우.
400	오류	ERR-4504	암호화에 실패했을 경우.
400	오류	ERR-4506	전송받은 이미지 의 byte정보를 추출하지 못했을 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5009	인식 엔진 - 최대 얼굴 검출 개수 설정 에러
500	오류	ERR-5010	인식 엔진 - 얼굴 검출 최소/최대 사이즈 설정 에러
500	오류	ERR-5011	인식 엔진 - 얼굴 검출 중 에러
500	오류	ERR-5013	인식 엔진 - 정렬된 얼굴 이미지 추출 중 에러

Status Code	Type	return_code	Description
500	오류	ERR-5014	인식 엔진 - 정렬된 얼굴 이미지 추출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

얼굴 검출 결과는 박스 정보(x, y, width, height) 만 제공되며, 정렬된 얼굴 이미지의 사이즈 정보와 base64로 인코딩된 이미지 데이터가 함께 전달됩니다.

항목	타입	설명
count	Integer	검출된 얼굴 개수
aligned_faces	JSON Array	얼굴 검출 및 각각의 정렬된 이미지 결과 (요청 및 응답 예시 참조)
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

```
curl -X POST http://127.0.0.1:{port}/alignedfaces/v2?encrypt_mode=false --
form 'image=@image.jpg'
{
  "count": 5,
  "aligned_faces": [
    {
      "box": {
        "x": 117.23624,
        "y": 247.9227,
        "width": 202.05891,
        "height": 202.05891
      },
      "image_height": 112,
      "image_width": 112,
      "image_channel": 3,
```

```

    "image_data": "SVlgTVphS1dfTVxNr..." # base64 인코딩 얼굴 이미지
데이터
    },
    ...
  ],
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}

```

```

curl -X POST http://127.0.0.1:{port}/alignedfaces/v2?encrypt_mode=true --
form 'encrypted_image=@encrypted.jpg'
{
  "count": 1,
  "aligned_faces": [
    {
      "box": {
        "x": 117.23624,
        "y": 247.9227,
        "width": 202.05891,
        "height": 202.05891
      },
      "image_height": 150,
      "image_width": 150,
      "image_channel": 3,
      "image_data": "SVlgTVphS1dfTVxNr..." # base64 인코딩 얼굴 이미지
데이터
    }
  ],
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}

```

- ERR-4501

```

curl -X POST http://127.0.0.1:{port}/alignedfaces/v2?encrypt_mode=false --
form 'image=@image.jpg'
{
  "return_msg": {
    "return_code": "ERR-4501",
    "return_msg": "Maximum upload size exceeded (maximum: 20MB)"
  }
}

```

4.14. POST /alignedfaces/metatemplate

여러 개의 얼굴이 포함된 이미지를 전달하여, 이미지 내의 얼굴 검출 결과와 각각의 정렬된 얼굴 이미지를 동시에 얻어냅니다. 얼굴 검출의 최소 및 최대 사이즈, 최대 검출 갯수를 각각 지정할 수 있습니다. 따로 지정하지 않을 경우 이하에 명시된 기본값을 적용해 얼굴을 검출합니다. **112 x 112 x 3 사이즈의 JPEG2000(jp2) 포맷 RGB 영상**의 정렬된 얼굴 이미지가 **base64 인코딩**되어 반환됩니다.

Request Query

항목	설명	예시
minsize	최소 얼굴 검출 사이즈	/alignedfaces/metatemplate?minsize=48
maxsize	최대 얼굴 검출 사이즈	/alignedfaces/metatemplate?maxsize=150
detectcount	최대 얼굴 검출 갯수	/alignedfaces/metatemplate?detectcount=5
encrypt_mode	암호화 모드 사용 유무 (디폴트 false)	/alignedfaces/metatemplate?encrypt_mode=true

minsize가 명시되지 않은 경우 24을 기본값으로 사용합니다. maxsize가 요청 이미지보다 클 경우 이미지의 크기를 maxsize로 설정합니다. detectcount가 명시되지 않은 경우 10을 기본값으로 설정합니다.

Request Header

항목	내용	비고
Content-type	image/jpeg	MIME 타입이 명시되지 않은 경우 image/jpeg를 기본값으로 사용함

Request Body

항목	타입	필수여부	설명
-	JPEG 파일 데이터	Y	얼굴 검출 및 특징점 추출을 수행할 이미지

Response Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
400	오류	ERR-4002	HTTP Request의 URI 쿼리 실패
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러

Status Code	Type	return_code	Description
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5009	인식 엔진 - 최대 얼굴 검출 개수 설정 에러
500	오류	ERR-5010	인식 엔진 - 얼굴 검출 최소/최대 사이즈 설정 에러
500	오류	ERR-5011	인식 엔진 - 얼굴 검출 중 에러
500	오류	ERR-5013	인식 엔진 - 정렬된 얼굴 이미지 추출 중 에러
500	오류	ERR-5014	인식 엔진 - 정렬된 얼굴 이미지 추출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
503	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

얼굴 검출 결과는 박스 정보(x, y, width, height) 만 제공되며, 정렬된 얼굴 이미지의 사이즈 정보와 base64로 인코딩된 이
미지 데이터가 함께 전달됩니다.

항목	타입	설명
count	Integer	검출된 얼굴 개수
aligned_faces	JSON Array	얼굴 검출 및 각각의 정렬된 이미지 결과 (요청 및 응답 예시 참조)
return_msg	JSON Object	결과 응답 코드와 메시지 (3. 응답 코드 목록 참조)

요청 및 응답 예시

```
curl -X POST http://127.0.0.1:{port}/alignedfaces/metatemplate --data-  
binary '@test.jpg'  
{  
  "count": 5,  
  "aligned_faces": [  
    {  
      "box": {
```



```
        "x": 117.23624,
        "y": 247.9227,
        "width": 202.05891,
        "height": 202.05891
    },
    "image_height": 112,
    "image_width": 112,
    "image_channel": 3,
    "image_data": "SVlgTVphS1dfTVxNr..." # base64 인코딩 얼굴 이미지
데이터(jp2 image format)
    },
    ...
],
"return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
}
}
```

4.15. POST /croppedface

API 개요

얼굴 이미지를 전송하여 얼굴 영역을 검출하고 검출된 얼굴 영역에서 ratio 비율만큼 상,하,좌,우로 확대한 영역으로 크롭 됩니다. Query String 값인 ratio로 크롭할 영역의 비율을 지정할 수 있고 resize_width를 통해 반환될 이미지의 너비를 조정할 수 있습니다.
그리고 크롭한 얼굴 이미지를 base64 인코딩 형식으로 반환합니다.

Request Query

항목	필수여부	설명
ratio	Y	크롭 할 영역의 비율 지정
resize_width	Y	사진의 너비 사이즈
encrypt_mode	N	암호화 모드 사용 유무 (디폴트 false)

금융결제원 안면인증공동시스템 기준 예시

- 신분증상 얼굴사진 크롭시
 - ratio : **0.25**
 - resize_width : **150**
- 셀피 얼굴사진 크롭시
 - ratio : **0.33**
 - resize_width : **200**

Request Header

항목	내용	비고
Content-type	image/jpeg	MIME 타입이 명시되지 않은 경우 image/jpeg를 기본값으로 사용함

Request Body

항목	타입	필수여부	설명
-	JPEG 파일 데이터	Y	얼굴 검출 및 특징점 추출을 수행할 이미지

Response Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
200	오류	FAIL-0000	얼굴 검출 실패
400	오류	ERR-4002	HTTP Request의 URI 쿼리 실패
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	오류	ERR-4503	암호화 된 정보를 해독하지 못했을 경우.
400	오류	ERR-4504	암호화에 실패했을 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5011	인식 엔진 - 얼굴 검출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
503	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
----	----	----

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

항목	타입	설명
image_height	Integer	crop된 얼굴 이미지의 높이
image_width	Integer	crop된 얼굴 이미지의 너비
image_data	String	crop된 얼굴 이미지 데이터가 base64 인코딩 형식으로 반환
return_msg	JSON Object	결과 응답 코드와 메시지 (3. 응답 코드 목록 참조)

요청 및 응답 예시

• SUCC-0000

- 신분증상 얼굴 사진 크롭

```
curl -X POST 'http://127.0.0.1:{port}/croppedface?ratio=0.2&resize_width=150&encrypt_mode=true' -H 'Content-Type: image/jpeg' --data-binary '@test.jpg'
{
  "image_height": 188,
  "image_width": 150,
  "image_data": "SVlgTVphS1dfTVxNr...", # base64 인코딩 얼굴 이미지 데이터
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

- 셀피 얼굴 사진 크롭시

```
curl -X POST 'http://127.0.0.1:{port}/croppedface?ratio=0.33&resize_width=200' -H 'Content-Type: image/jpeg' --data-binary '@test.jpg'
{
  "image_height": 295,
  "image_width": 200,
  "image_data": "SVlgTVphS1dfTVxNr...", # base64 인코딩 얼굴 이미지 데이터
  "return_msg": {
    "return_code": "SUCC-0000",
  }
}
```

```
        "return_msg": "OK."
    }
}
```

• FAIL-0000

```
curl -X POST 'http://127.0.0.1:{port}/croppedface?
ratio=0.33&resize_width=200' -H 'Content-Type: image/jpeg' --data-
binary '@test.jpg'
{
    "image_height": 0,
    "image_width": 0,
    "image_data": null,
    "return_msg": {
        "return_code": "FAIL-0000",
        "return_msg": "face not found."
    }
}
```

4.16. GET /compare/validation-config

API 개요

얼굴 비교의 validation 로직에서 사용하는 수치 값들의 목록을 반환합니다.

Request Query

Request Header

항목	내용	비고
Content-type	application/json	

Request Body

Response Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

항목	타입	설명
compare_validation	JSON Object	얼굴 비교에 사용되는 validation 수치 값 목록 (요청 및 응답 예시 참조)
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

- SUCC-0000

```
curl -X GET http://127.0.0.1:{port}/compare/validation-config
{
  "compare_validation": {
    "mask_confidence": 0.5,
    "max_face_size_rate": 1.0,
    "min_face_size_rate": 0.25,
    "max_pitch_degree": 25.0,
    "min_pitch_degree": -10.0,
    "max_yaw_degree": 20.0,
    "min_yaw_degree": -20.0,
    "max_roll_degree": 20.0,
    "min_roll_degree": -20.0,
    "quality_confidence": 63.96,
    "landmark_confidence": 0.8
  },
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

4.17. POST /compare

API 개요

두 장의 **얼굴 이미지**를 전달하여, 두 얼굴의 매칭 결과를 얻어냅니다. 각각의 이미지에서 검출된 얼굴들 중 **가장 큰 얼굴**을 기준으로 계산하여 매칭 결과와 신뢰도 값을 리턴합니다. 이때 서버를 실행할 때 지정한 threshold 값을 기준으로 매칭 결과가 결정됩니다.

Request Query

Request Header

항목	내용	비고
Content-type	multipart/form-data	

Request Body

항목	타입	필수 여부	설명
image_a	JPEG 파일 데이터	N	첫 번째 얼굴 이미지 (encrypt_mode 미 사용 시 필수)
image_b	JPEG 파일 데이터	N	두 번째 얼굴 이미지 (encrypt_mode 미 사용 시 필수)
image_a_validate	String	N	첫 번째 얼굴 이미지 검증 여부(디폴트 'N')
image_b_validate	String	N	두 번째 얼굴 이미지 검증 여부(디폴트 'N')
image_a_extract_feature_base64	String	N	첫 번째 얼굴 이미지의 특징점 반환 여부(디폴트 'N')
image_b_extract_feature_base64	String	N	두 번째 얼굴 이미지의 특징점 반환 여부(디폴트 'N')
encrypt_mode	Boolean	N	암호화 모드 사용 유무 (디폴트 false)
encrypted_image_a	String 파일 데이터	N	첫 번째 얼굴 이미지 암호화 된 파일 (encrypt_mode 사용 시 필수)
encrypted_image_b	String 파일 데이터	N	두 번째 얼굴 이미지 암호화 된 파일 (encrypt_mode 사용 시 필수)
fin_code	String	N	매칭에 대한 개별 임계치를 사용하는 경우에 설정한 임계치의 식별코드 (백오피스를 사용하는 경우 설정된 기관 코드, 미사용시 fin_code: "000", fin_name: "default")

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
200	실패	FAIL-A000	얼굴 비교 실패 - a 이미지에서 얼굴 검출 실패
200	실패	FAIL-B000	얼굴 비교 실패 - b 이미지에서 얼굴 검출 실패
200	실패	FAIL-A001	얼굴 비교 실패 - a 이미지의 얼굴 크기가 너무 작음
200	실패	FAIL-B001	얼굴 비교 실패 - b 이미지의 얼굴 크기가 너무 작음
200	실패	FAIL-A002	얼굴 비교 실패 - a 이미지의 얼굴 크기가 너무 큼
200	실패	FAIL-B002	얼굴 비교 실패 - b 이미지의 얼굴 크기가 너무 큼
200	실패	FAIL-A003	얼굴 비교 실패 - a 이미지의 얼굴이 정면을 바라보고 있지 않음
200	실패	FAIL-B003	얼굴 비교 실패 - b 이미지의 얼굴이 정면을 바라보고 있지 않음

Status Code	Type	return_code	Description
200	실패	FAIL-A004	얼굴 비교 실패 - a 이미지의 얼굴에서 마스크 착용이 감지됨
200	실패	FAIL-B004	얼굴 비교 실패 - b 이미지의 얼굴에서 마스크 착용이 감지됨
200	실패	FAIL-A005	얼굴 비교 실패 - a 이미지의 얼굴 검출 신뢰도가 낮음
200	실패	FAIL-B005	얼굴 비교 실패 - b 이미지의 얼굴 검출 신뢰도가 낮음
200	실패	FAIL-A006	얼굴 비교 실패 - a 이미지의 얼굴 적합성 점수가 낮음
200	실패	FAIL-B006	얼굴 비교 실패 - b 이미지의 얼굴 적합성 점수가 낮음
400	오류	ERR-4002	HTTP Request의 URI 쿼리 실패
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	오류	ERR-4501	wrapper server form 전송 데이터 20MB 초과
400	오류	ERR-4503	암호화 된 정보를 해독하지 못했을 경우.
400	오류	ERR-4504	암호화에 실패했을 경우.
400	오류	ERR-4506	전송받은 이미지 의 byte정보를 추출하지 못했을 경우.
400	오류	ERR-4507	설정 정보에 대한 검증을 통과 하지 못한 파일의 경우 발생하는 에러.
400	오류	ERR-4508	금융사의 config 사용이 활성화 되지 않은 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5009	인식 엔진 - 최대 얼굴 검출 개수 설정 에러
500	오류	ERR-5010	인식 엔진 - 얼굴 검출 최소/최대 사이즈 설정 에러
500	오류	ERR-5011	인식 엔진 - 얼굴 검출 중 에러
500	오류	ERR-5012	인식 엔진 - 얼굴 특징점 추출 중 에러
500	오류	ERR-5014	인식 엔진 - 정렬된 얼굴 이미지 추출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5018	유사도 계산 중 에러
500	오류	ERR-5021	인식 엔진 - 얼굴 마스크 착용여부 확인 중 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러

Status Code	Type	return_code	Description
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

매칭 결과는 사전에 설정된 threshold를 기준으로 계산되며 true/false 값으로 반환됩니다. 매칭 신뢰도는 0~1 사이의 실수 값이며, 1에 가까울수록 동일 얼굴일 확률이 높습니다.

항목	타입	설명
similarity_confidence	Float	두 얼굴의 유사도를 수치화한 신뢰도
match_result	Integer	두 얼굴의 매칭 결과에 따른 심사 결과 (1: 자동승인, 2: 수동심사, 3: 자동 거절)
threshold_info	JSON Object	결과에 따른 심사 기준 값
image_a_feature_base64	String	첫 번째 얼굴 이미지의 특징점 결과를 base64 형태로 반환 (image_a_feature_base64="Y" 인 경우에만)
image_b_feature_base64	String	두 번째 얼굴 이미지의 특징점 결과를 base64 형태로 반환 (image_b_feature_base64="Y" 인 경우에만)
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

- SUCC-0000

```
curl -X POST http://127.0.0.1:{port}/compare --form
'image_a=@feature_test_a.jpg' --form 'image_b=@feature_test_b.jpg'
{
  "similarity_confidence": 0.997386,
  "match_result": 1,
  "threshold_info": {
    "fin_code": "000",
    "fin_name": "default",
    "auto_approve": {
```



```

        "min": 0.9998,
        "max": 1.0
    },
    "manual_review": {
        "min": 0.9972,
        "max": 0.9997
    },
    "auto_reject": {
        "min": 0.0,
        "max": 0.9971
    }
},
"return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
}
}

```

- SUCC-0000 (암호화 모드)

```

curl -X POST http://127.0.0.1:{port}/compare?encrypt_mode=true --form
'encrypted_image_a=@encrypted_a.jpg' --form
'encrypted_image_b=@encrypted_b.jpg'
{
    "similarity_confidence": 0.997386,
    "match_result": 1,
    "threshold_info": {
        "fin_code": "000",
        "fin_name": "default",
        "auto_approve": {
            "min": 0.9998,
            "max": 1.0
        },
    },
    "manual_review": {
        "min": 0.9972,
        "max": 0.9997
    },
    "auto_reject": {
        "min": 0.0,
        "max": 0.9971
    }
},
"return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
}
}

```

```

curl -X POST http://127.0.0.1:{port}/compare
--form 'image_a=@feature_test_a.jpg' \
--form 'image_a_validate="Y"' \
--form 'image_a_extract_feature_base64="Y"' \
--form 'image_b=@feature_test_b.jpg' \
--form 'image_b_validate="Y"' \
--form 'image_b_extract_feature_base64="N"'
{
  "similarity_confidence": 1,
  "match_result": 1,
  "threshold_info": {
    "fin_code": "000",
    "fin_name": "default",
    "auto_approve": {
      "min": 0.9998,
      "max": 1.0
    },
    "manual_review": {
      "min": 0.9972,
      "max": 0.9997
    },
    "auto_reject": {
      "min": 0.0,
      "max": 0.9971
    }
  },
  "image_a_feature_base64":
  "h2UVPchD67zSGWY8MTo4verz5Ty17CY9V1wDPQ...xKt4a9t1HovfIpHb0=",
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}

```

```

curl -X POST http://127.0.0.1:{port}/compare
--form 'image_a=@feature_test_a.jpg' \
--form 'image_a_validate="Y"' \
--form 'image_a_extract_feature_base64="N"' \
--form 'image_b=@feature_test_b.jpg' \
--form 'image_b_validate="Y"' \
--form 'image_b_extract_feature_base64="N"'
{
  "similarity_confidence": 1,
  "match_result": 1,
  "threshold_info": {
    "fin_code": "000",
    "fin_name": "default",
    "auto_approve": {
      "min": 0.9998,
      "max": 1.0
    },
  },

```

```
        "manual_review": {
            "min": 0.9972,
            "max": 0.9997
        },
        "auto_reject": {
            "min": 0.0,
            "max": 0.9971
        }
    },
    "return_msg": {
        "return_code": "SUCC-0000",
        "return_msg": "OK."
    }
}
```

- FAIL-A001

```
curl -X POST http://127.0.0.1:{port}/compare
--form 'image_a=@feature_test_a.jpg' \
--form 'image_a_validate="Y"' \
--form 'image_b=@feature_test_b.jpg' \
--form 'image_b_validate="N"'
{
    "similarity_confidence": 0.0,
    "match_result": 3,
    "threshold_info": {
        "fin_code": "000",
        "fin_name": "default",
        "auto_approve": {
            "min": 0.9998,
            "max": 1.0
        },
        "manual_review": {
            "min": 0.9972,
            "max": 0.9997
        },
        "auto_reject": {
            "min": 0.0,
            "max": 0.9971
        }
    },
    "return_msg": {
        "return_code": "FAIL-A001",
        "return_msg": "image 'a' face size is too small"
    }
}
```

- FAIL-A004

```
curl -X POST http://127.0.0.1:{port}/compare
--form 'image_a=@feature_test_a.jpg' \
--form 'image_a_validate="Y"' \
--form 'image_b=@feature_test_b.jpg' \
--form 'image_b_validate="N"'
{
  "similarity_confidence": 0.0,
  "match_result": 3,
  "threshold_info": {
    "fin_code": "000",
    "fin_name": "default",
    "auto_approve": {
      "min": 0.9998,
      "max": 1.0
    },
    "manual_review": {
      "min": 0.9972,
      "max": 0.9997
    },
    "auto_reject": {
      "min": 0.0,
      "max": 0.9971
    }
  },
  "return_msg": {
    "return_code": "FAIL-A004",
    "return_msg": "mask wearing detected on the face of image 'a'"
  }
}
```

- ERR-4501

```
curl -X POST http://127.0.0.1:{port}/alignedfaces/v2?encrypt_mode=false --
form 'image=@image.jpg'
{
  "return_msg": {
    "return_code": "ERR-4501",
    "return_msg": "Maximum upload size exceeded (maximum: 20MB)"
  }
}
```

4.18. POST /compare/feature

API 개요

두 개의 얼굴 특징점을 전달하여, 두 얼굴의 매칭 결과와 신뢰도 값을 리턴합니다. 이때 서버를 실행할 때 지정한 threshold 값을 기준으로 매칭 결과가 결정됩니다.

Request Query

Request Header

항목	내용	비고
Content-type	multipart/form-data	

Request Body

항목	타입	필수 여부	설명
feature_a	String	Y	base64로 인코딩 된 첫 번째 얼굴 특징점
feature_b	String	Y	base64로 인코딩 된 두 번째 얼굴 특징점
fin_code	String	N	매칭에 대한 개별 임계치를 사용하는 경우에 설정한 임계치의 식별코드 (백오피스를 사용하는 경우 설정된 기관 코드, 미사용시 fin_code: "000", fin_name: "default")

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
400	오류	ERR-4006	특징점 비교 실패 - a 특징점의 형식이 부적절함
400	오류	ERR-4007	특징점 비교 실패 - b 특징점의 형식이 부적절함
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	오류	ERR-4501	wrapper server form 전송 데이터 20MB 초과
400	오류	ERR-4508	금융사의 config 사용이 활성화 되지 않은 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

매칭 결과는 사전에 설정된 threshold를 기준으로 계산되며 true/false 값으로 반환됩니다. 매칭 신뢰도는 0~1 사이의 실수 값이며, 1에 가까울수록 동일 얼굴일 확률이 높습니다.

항목	타입	설명
similarity_confidence	Float	두 얼굴의 유사도를 수치화한 신뢰도
match_result	Integer	두 얼굴의 매칭 결과에 따른 심사 결과 (1: 자동승인, 2: 수동심사, 3: 자동거절)
threshold_info	JSON Object	결과에 따른 심사 기준 값
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

- SUCC-0000

```
curl -X POST http://127.0.0.1:{port}/compare/feature \
--form 'feature_a="base64 encoded feature"' \
--form 'feature_b="base64 encoded feature"'
{
  "similarity_confidence": 0.997386,
  "match_result": 1,
  "threshold_info": {
    "fin_code": "000",
    "fin_name": "default",
    "auto_approve": {
      "min": 0.9998,
      "max": 1.0
    },
    "manual_review": {
      "min": 0.9972,
      "max": 0.9997
    },
    "auto_reject": {
      "min": 0.0,
      "max": 0.9971
    }
  }
}
```

```

    },
    "return_msg": {
      "return_code": "SUCC-0000",
      "return_msg": "OK."
    }
  }
}

```

- ERR-4006

```

curl -X POST http://127.0.0.1:{port}/compare/feature \
--form 'feature_a="invalid base64 encoded feature"' \
--form 'feature_b="base64 encoded feature"'
{
  "similarity_confidence": 0.0,
  "match_result": 3,
  "threshold_info": {
    "fin_code": "000",
    "fin_name": "default",
    "auto_approve": {
      "min": 0.9998,
      "max": 1.0
    },
  },
  "manual_review": {
    "min": 0.9972,
    "max": 0.9997
  },
  "auto_reject": {
    "min": 0.0,
    "max": 0.9971
  },
  },
  "return_msg": {
    "return_code": "ERR-4006",
    "return_msg": "feature 'a' is invalid format"
  }
}

```

- ERR-4007

```

curl -X POST http://127.0.0.1:{port}/compare/feature \
--form 'feature_a="base64 encoded feature"' \
--form 'feature_b="invalid base64 encoded feature"'
{
  "similarity_confidence": 0.0,
  "match_result": 3,
  "threshold_info": {
    "fin_code": "000",
    "fin_name": "default",
    "auto_approve": {

```

```

        "min": 0.9998,
        "max": 1.0
    },
    "manual_review": {
        "min": 0.9972,
        "max": 0.9997
    },
    "auto_reject": {
        "min": 0.0,
        "max": 0.9971
    }
},
"return_msg": {
    "return_code": "ERR-4007",
    "return_msg": "feature 'b' is invalid format"
}
}

```

- ERR-4501

```

curl -X POST http://127.0.0.1:{port}/alignedfaces/v2?encrypt_mode=false --
form 'image=@image.jpg'
{
    "return_msg": {
        "return_code": "ERR-4501",
        "return_msg": "Maximum upload size exceeded (maximum: 20MB)"
    }
}

```

4.19. POST /masks

API 개요

여러 개의 얼굴이 포함된 이미지를 전달하여, 이미지 내에서 얼굴을 검출한 결과 및 각 얼굴의 마스크 착용 여부에 대한 신뢰도를 얻어냅니다. 또한 입력 이미지에 얼굴이 없을 경우, 결과 얼굴 count가 0인 것이 정상 동작입니다. 실제로 얼굴을 잡지 못했으니 오동작이 아닙니다.

Request Query

Request Header

항목	내용	비고
Content-type	image/jpeg	MIME 타입이 명시되지 않은 경우 image/jpeg를 기본값으로 사용함

Request Body

항목	타입	필수여부	설명
-	JPEG 파일 데이터	Y	얼굴 검출 및 마스크 착용여부를 판단할 이미지

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
200	실패	FAIL-0000	얼굴 검출 실패
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	오류	ERR-4504	암호화에 실패했을 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5009	인식 엔진 - 최대 얼굴 검출 개수 설정 에러
500	오류	ERR-5010	인식 엔진 - 얼굴 검출 최소/최대 사이즈 설정 에러
500	오류	ERR-5011	인식 엔진 - 얼굴 검출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5021	인식 엔진 - 얼굴 마스크 착용여부 확인 중 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

얼굴 검출 결과에는 박스 정보 (x, y, width, height), 106점 랜드마크 정보, 얼굴 각도(Yaw, Pitch, Roll) 정보가 포함됩니다. 마스크 착용 여부 확인 결과는 신뢰도로 표현되며, 0~1 사이의 실수 값으로 표현됩니다. 1에 가까울수록 착용하였을 확률이 높다는 의미입니다.

항목	타입	설명
count	Integer	검출된 얼굴 개수
mask_results	JSON Object	얼굴 검출 및 마스크 착용 여부 확인 결과 (요청 및 응답 예시 참조)
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

```
curl -X POST http://127.0.0.1:{port}/masks -H 'Content-Type: image/jpeg' -
-data-binary '@test.jpg'
{
  "count": 5,
  "mask_results": [
    {
      "box": {
        "x": 117.23624,
        "y": 247.9227,
        "width": 202.05891,
        "height": 202.05891
      },
      "confidence": 0.9992258
    },
    ...
  ],
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

4.20. GET /liveness/validation-config

API 개요

POST /liveness API의 face validation(best shot 검증)로직에 사용하는 수치 값들의 목록을 반환합니다.

Request Query

Request Header

항목	내용	비고
Content-type	application/json	

Request Body

Response Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

항목	타입	설명
liveness_validation	JSON Object	라이브니스에 사용되는 validation 수치 값 목록 (요청 및 응답 예시 참조)
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

- SUCC-0000

```
curl -X GET http://127.0.0.1:{port}/liveness/validation-config
{
  "liveness_validation": {
    "mask_confidence": 0.5,
    "max_face_size_rate": 0.6,
    "min_face_size_rate": 0.4,
    "face_position_rate": 10.0,
    "max_pitch_degree": 20.0,
    "min_pitch_degree": -10.0,
    "max_yaw_degree": 10.0,
    "min_yaw_degree": -10.0,
    "max_roll_degree": 10.0,
    "min_roll_degree": -10.0,
    "quality_confidence": 63.96,
    "landmark_confidence": 0.9
  },
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

4.21. POST /liveness

API 개요

한 장의 이미지를 보내서 얼굴 진위 여부를 확인합니다.

Request Query

Request Header

항목	내용	비고
Content-type	multipart/form-data	

Request Body

항목	타입	필수 여부	설명
image	JPEG 파일 데이터	N	Liveness를 측정할 이미지 (encrypt_mode 미 사용 시 필수)
encrypt_mode	Boolean	N	암호화 모드 사용 유무 (디폴트 false)
encrypted_image	String 파일 데이터	N	Liveness를 측정할 이미지 암호화 된 파일 (encrypt_mode 사용 시 필수)
compare_image	JPEG 파일 데이터	N	Liveness와 매칭을 동시에 진행하고 싶을 경우 비교에 필요한 이미지 (encrypt_mode 미 사용 시)
encrypted_compare_image	String 파일 데이터	N	Liveness와 매칭을 동시에 진행하고 싶을 경우 비교에 필요한 이미지 암호화 된 파일 (encrypt_mode 사용 시)
image_a_validate	String	N	compare 기능 추가 사용 시 매칭 이미지(compare_image or encrypted_compare_image) 검증 여부(디폴트 'Y'), 유효하지 않는 값 (디폴트 'N')
image_b_validate	String	N	compare 기능 추가 사용 시 라이브니스 성공 이미지 검증 여부(디폴트 'Y'), 유효하지 않는 값 (디폴트 'N')
extract_feature_base64	Boolean	N	Liveness 검증을 완료한 이미지 하나에 대한 특징점 벡터값 추가 제공 유무 (byte 값을 base64로 제공)

항목	타입	필수여부	설명
fin_code	String	N	매칭에 대한 개별 임계치를 사용하는 경우에 설정한 임계치의 식별 코드 (백오피스를 사용하는 경우 설정된 기관 코드, 미사용시 fin_code: "000", fin_name: "default")

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
200	실패	FAIL-0000	얼굴 검출 실패
200	실패	FAIL-0001	베스트 샷 추출 실패 - 이미지의 얼굴 크기가 너무 작음
200	실패	FAIL-0002	베스트 샷 추출 실패 - 이미지의 얼굴 크기가 너무 큼
200	실패	FAIL-0003	베스트 샷 추출 실패 - 이미지의 얼굴이 정면을 바라보고 있지 않음
200	실패	FAIL-0004	베스트 샷 추출 실패 - 이미지의 얼굴에서 마스크 착용이 감지됨
200	실패	FAIL-0005	베스트 샷 추출 실패 - 이미지의 얼굴의 랜드마크 점수가 낮음
200	실패	FAIL-0006	베스트 샷 추출 실패 - 이미지에서 얼굴이 가운데에 위치하고 있지 않음
200	실패	FAIL-0007	베스트 샷 추출 실패 - 이미지에서 얼굴 적합성 점수가 낮음
200	실패	FAIL-0008	베스트 샷 추출 실패 - 이미지에서 얼굴이 아래를 바라보고 있음
200	실패	FAIL-0009	베스트 샷 추출 실패 - 이미지에서 얼굴이 위를 바라보고 있음
200	실패	FAIL-0010	베스트 샷 추출 실패 - 이미지에서 얼굴이 기울어져 있음
200	실패	FAIL-0011	베스트 샷 추출 실패 - 이미지에서 얼굴의 적합성 점수가 위조판별을 위한 기준보다 낮음
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	오류	ERR-4501	wrapper server form 전송 데이터 20MB 초과
400	오류	ERR-4503	암호화 된 정보를 해독하지 못했을 경우.
400	오류	ERR-4504	암호화에 실패했을 경우.

Status Code	Type	return_code	Description
400	오류	ERR-4506	전송받은 이미지 의 byte정보를 추출하지 못했을 경우.
400	오류	ERR-4508	금융사의 config 사용이 활성화 되지 않은 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5009	인식 엔진 - 최대 얼굴 검출 개수 설정 에러
500	오류	ERR-5010	인식 엔진 - 얼굴 검출 최소/최대 사이즈 설정 에러
500	오류	ERR-5011	인식 엔진 - 얼굴 검출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5019	인식 엔진 - 얼굴 진위 여부 확인 중 에러
500	오류	ERR-5021	인식 엔진 - 얼굴 마스크 착용여부 확인 중 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5025	얼굴 퀄리티 확인을 하는 과정에서의 에러
500	오류	ERR-5026	얼굴 랜드마크 점수를 얻는 과정에서의 에러
500	오류	ERR-5027	위변조판별을 위한 얼굴 적합성 점수를 확인하는 과정에서의 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러
500	오류	ERR-5501	go server 와 통신중 예상하지 못한 에러가 발생한 경우.
500	오류	ERR-5502	request URI 가 go server 에 존재 하지 않는 경우.

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

신뢰도(confidence)는 0~1 사이의 실수 값이며, 1에 가까울수록 실제 얼굴(Live)입니다.

항목	타입	설명
----	----	----

항목	타입	설명
confidence	Float	liveness 정도 (요청 및 응답 예시 참조)
is_live	Boolean	결과 값에 따른 심사 결과 (true: 자동승인, false: 자동거절)
threshold_info	JSON Object	결과에 따른 심사 기준 값
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)
compare_result	JSON Object	추가 매칭 결과 응답 코드와 메시지 (POST /compare 문서 참조)
feature_base64	JSON Object	추가 백터값 추출 결과 Byte를 base64 encoding한 값

요청 및 응답 예시

```
curl -X POST http://127.0.0.1:{port}/liveness?encrypt_mode=false --form
'image=@image.jpg'
{
  "confidence": 0.74912024,
  "is_live": false,
  "threshold_info": {
    "fin_code": "000",
    "fin_name": "default",
    "auto_approve": {
      "min": 0.8836,
      "max": 1.0
    },
    "auto_reject": {
      "min": 0.0,
      "max": 0.8835
    }
  },
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

```
curl -X POST http://127.0.0.1:{port}/liveness?encrypt_mode=true --form
'encrypted_image=@encrypted_image.jpg'
{
  "confidence": 0.74912024,
  "is_live": false,
  "threshold_info": {
    "fin_code": "000",
    "fin_name": "default",
    "auto_approve": {
      "min": 0.8836,
      "max": 1.0
    }
  }
}
```

```

    },
    "auto_reject": {
      "min": 0.0,
      "max": 0.8835
    }
  },
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}

```

compare / feature 기능 사용

```

curl -X POST http://127.0.0.1:{port}/liveness?encrypt_mode=true
--form 'encrypted_image=@encrypted_image.jpg' \
--form 'encrypted_compare_image=@encrypted_compare_image' \
--form 'extract_feature_base64="true"'
{
  "confidence": 0.99462026,
  "is_live": true,
  "threshold_info": {
    "fin_code": "000",
    "fin_name": "default",
    "auto_approve": {
      "min": 0.7037,
      "max": 1.0
    }
  },
  "auto_reject": {
    "min": 0.0,
    "max": 0.7036
  }
},
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  },
  "compare_result": {
    "similarity_confidence": 1.0,
    "match_result": 1,
    "threshold_info": {
      "fin_code": "000",
      "fin_name": "default",
      "auto_approve": {
        "min": 0.9998,
        "max": 1.0
      }
    },
    "manual_review": {
      "min": 0.9985,
      "max": 0.9997
    },
    "auto_reject": {

```



```
        "min": 0.0,
        "max": 0.9984
    },
    },
    "return_msg": {
        "return_code": "SUCC-0000",
        "return_msg": "OK."
    }
},
"feature_base64": {
    "value": "h2UVPchD67zSGWY8MTo4verz5Ty17CY9V1wDPQB...",
    "return_msg": {
        "return_code": "SUCC-0000",
        "return_msg": "OK."
    }
}
}
```

- ERR-4501

```
curl -X POST http://127.0.0.1:{port}/alignedfaces/v2?encrypt_mode=false --
form 'image=@image.jpg'
{
    "return_msg": {
        "return_code": "ERR-4501",
        "return_msg": "Maximum upload size exceeded (maximum: 20MB)"
    }
}
```

4.22. POST /liveness/multiframe

API 개요

순차적으로 찍은 **4장**의 이미지를 하나의 압축파일로 압축하여 보내서 얼굴 진위 여부(Multi frame Liveness)를 확인합니다.
반드시 **동일인물의 이미지 4장**이 필요합니다.

Request Query

Request Header

항목	내용	비고
Content-type	multipart/form-data	

Request Body

항목	타입	필수 여부	설명
images	zip 압축 파일	N	Liveness를 측정할 4장의 이미지를 압축한 zip 파일 (encrypt_mode 미 사용 시 필수)
encrypt_mode	Boolean	N	암호화 모드 사용 유무 (디폴트 false)
encrypted_images	String 파일 데이터	N	Liveness를 측정할 4장의 이미지를 압축한 zip 파일을 암호화 된 파일 (encrypt_mode 사용 시 필수)
compare_image	JPEG 파일 데이터	N	Liveness와 매칭을 동시에 진행하고 싶을 경우 비교에 필요한 이미지 (encrypt_mode 미 사용 시)
encrypted_compare_image	String 파일 데이터	N	Liveness와 매칭을 동시에 진행하고 싶을 경우 비교에 필요한 이미지 암호화 된 파일 (encrypt_mode 사용 시)
image_a_validate	String	N	compare 기능 추가 사용 시 매칭 이미지(compare_image or encrypted_compare_image) 검증 여부(디폴트 'Y'), 유효하지 않는 값 (디폴트 'N')
image_b_validate	String	N	compare 기능 추가 사용 시 라이브니스 성공 이미지 검증 여부(디폴트 'Y'), 유효하지 않는 값 (디폴트 'N')
extract_feature_base64	Boolean	N	Liveness 검증을 완료한 이미지 하나에 대한 특징점 벡터값 추가 제공 유무 (byte 값을 base64로 제공)
fin_code	String	N	매칭에 대한 개별 임계치를 사용하는 경우에 설정한 임계치의 식별 코드 (백오피스를 사용하는 경우 설정된 기관 코드, 미사용시 fin_code: "000", fin_name: "default")

- 요청 이미지 4장을 zip 포맷으로 압축하여, 압축 파일을 전송해야 합니다. 압축파일의 이름은 상관 없습니다.
- 요청 이미지들은 png / jpg 포맷을 지원하며, 각각의 이름은 1, 2, 3, 4여야 합니다.
- **이미지 이름의 숫자가 처리 순서**가 됩니다. 시간 순서가 있는 이미지들일 경우 파일명을 순서대로 설정하여 압축해주셔야 합니다.

일례로, 다음과 같은 내용을 포함한 zip 파일은 전송 가능합니다.

```
images1.zip (1.png, 2.png, 3.jpg, 4.jpg)
images2.zip (3.jpg, 4.jpg, 1.jpg, 2.png)
```

그러나 다음과 같이 인식할 수 없는 이름이거나, 이미지 갯수가 4장이 아닌 경우 전송이 불가능합니다.

images3.zip (1.png, img2.jpg, test3.jpg, 4.jpg)
images4.zip (1.jpg, 2.jpg, 3.jpg)

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
200	실패	FAIL-0000	얼굴 검출 실패
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
400	오류	ERR-4022	인식 엔진 - Multi frame liveness 요청의 zip 압축파일을 해제할 수 없음
400	오류	ERR-4023	인식 엔진 - Multi frame liveness 요청 압축 파일 내 이미지 획득할 수 없음
400	오류	ERR-4024	인식 엔진 - Multi frame liveness 요청 압축 파일 내 이미지 갯수가 다름
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	오류	ERR-4501	wrapper server form 전송 데이터 20MB 초과
400	오류	ERR-4503	암호화 된 정보를 해독하지 못했을 경우.
400	오류	ERR-4504	암호화에 실패했을 경우.
400	오류	ERR-4505	서버에서 전송받은 압축파일은 압축해제 실패시 에러
400	오류	ERR-4508	금융사의 config 사용이 활성화 되지 않은 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5009	인식 엔진 - 최대 얼굴 검출 개수 설정 에러
500	오류	ERR-5010	인식 엔진 - 얼굴 검출 최소/최대 사이즈 설정 에러
500	오류	ERR-5011	인식 엔진 - 얼굴 검출 중 에러
500	오류	ERR-5012	인식 엔진 - 얼굴 특징점 추출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5019	인식 엔진 - 얼굴 진위 여부 확인 중 에러

Status Code	Type	return_code	Description
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

신뢰도(confidence)는 0~1 사이의 실수 값이며, 1에 가까울수록 실제 얼굴(Live)입니다.

항목	타입	설명
confidence	Float	liveness 정도 (요청 및 응답 예시 참조)
is_live	Boolean	결과 값에 따른 심사 결과 (true: 자동승인, false: 자동거절)
threshold_info	JSON Object	결과에 따른 심사 기준 값
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)
compare_result	JSON Object	추가 매칭 결과 응답 코드와 메시지 (POST /compare 문서 참조)
feature_base64	JSON Object	추가 벡터값 추출 결과 Byte를 base64 encoding한 값

요청 및 응답 예시

```
curl -X POST http://127.0.0.1:{port}/liveness/multiframe --form
'images=@multiframe.zip'
{
  "confidence": 0.74912024,
  "is_live": false,
  "threshold_info": {
    "fin_code": "000",
    "fin_name": "default",
    "auto_approve": {
      "min": 0.8836,
      "max": 1.0
    },
    "auto_reject": {
      "min": 0.0,
```

```

        "max": 0.8835
    },
    },
    "return_msg": {
        "return_code": "SUCC-0000",
        "return_msg": "OK."
    }
}

```

```

curl -X POST http://127.0.0.1:{port}/liveness/multiframe?encrypt_mode=true
--form 'encrypted_images=@encrypted_multiframe.zip'
{
    "confidence": 0.74912024,
    "is_live": false,
    "threshold_info": {
        "fin_code": "000",
        "fin_name": "default",
        "auto_approve": {
            "min": 0.8836,
            "max": 1.0
        },
    },
    "auto_reject": {
        "min": 0.0,
        "max": 0.8835
    },
    },
    "return_msg": {
        "return_code": "SUCC-0000",
        "return_msg": "OK."
    }
}

```

compare / feature 기능 사용

```

curl -X POST http://127.0.0.1:{port}/liveness/multiframe?encrypt_mode=true
--form 'encrypted_images=@"/encrypted_multiframe.zip"' \
--form 'encrypted_compare_image=@"encrypted_compare_image"' \
--form 'extract_feature_base64="true"'
{
    "confidence": 0.99462026,
    "is_live": true,
    "threshold_info": {
        "fin_code": "000",
        "fin_name": "default",
        "auto_approve": {
            "min": 0.7037,
            "max": 1.0
        },
    },
    "auto_reject": {

```

```

        "min": 0.0,
        "max": 0.7036
    },
    },
    "return_msg": {
        "return_code": "SUCC-0000",
        "return_msg": "OK."
    },
    "compare_result": {
        "similarity_confidence": 1.0,
        "match_result": 1,
        "threshold_info": {
            "fin_code": "000",
            "fin_name": "default",
            "auto_approve": {
                "min": 0.9998,
                "max": 1.0
            },
            "manual_review": {
                "min": 0.9985,
                "max": 0.9997
            },
            "auto_reject": {
                "min": 0.0,
                "max": 0.9984
            }
        },
        "return_msg": {
            "return_code": "SUCC-0000",
            "return_msg": "OK."
        }
    },
    "feature_base64": {
        "value": "h2UVPchD67zSGWY8MT04verz5Ty17CY9V1wDPQB...",
        "return_msg": {
            "return_code": "SUCC-0000",
            "return_msg": "OK."
        }
    }
}

```

- ERR-4501

```

curl -X POST http://127.0.0.1:{port}/alignedfaces/v2?encrypt_mode=false --
form 'image=@image.jpg'
{
    "return_msg": {
        "return_code": "ERR-4501",
        "return_msg": "Maximum upload size exceeded (maximum: 20MB)"
    }
}

```

4.23. POST /liveness/multiframe/v2

API 개요

순차적으로 찍은 4장 이상의 이미지를 하나의 압축파일로 압축하여 보내서 얼굴 진위 여부(Multi frame Liveness)를 확인합니다. 반드시 4장 이상의 동일인물의 이미지 및 동일한 파일 확장자의 이미지가 필요합니다. Liveness Mode가 **MULTI**로 설정된 서버에만 요청할 수 있습니다. Liveness Mode의 확인은 [GET /version](#) API를 참조하세요.

Request Query

Request Header

항목	내용	비고
Content-type	multipart/form-data	

Request Body

항목	타입	필수 여부	설명
images	zip 압축 파일	N	Liveness를 측정할 4장 이상의 이미지를 압축한 zip 파일 (encrypt_mode 미 사용 시 필수)
frame_counts	String	Y	Liveness를 측정할 이미지 파일의 개수
filename_extension	String	Y	Liveness를 측정할 이미지 파일의 확장자 이름
encrypt_mode	Boolean	N	암호화 모드 사용 유무 (디폴트 false)
encrypted_images	String 파일 데이터	N	Liveness를 측정할 4장의 이미지를 압축한 zip 파일을 암호화 된 파일 (encrypt_mode 사용 시 필수)
compare_image	JPEG 파일 데이터	N	Liveness와 매칭을 동시에 진행하고 싶을 경우 비교에 필요한 이미지 (encrypt_mode 미 사용 시)
encrypted_compare_image	String 파일 데이터	N	Liveness와 매칭을 동시에 진행하고 싶을 경우 비교에 필요한 이미지 암호화 된 파일 (encrypt_mode 사용 시)
image_a_validate	String	N	compare 기능 추가 사용 시 매칭 이미지(compare_image or encrypted_compare_image) 검증 여부(디폴트 'Y'), 유효하지 않는 값 (디폴트 'N')
image_b_validate	String	N	compare 기능 추가 사용 시 라이브니스 성공 이미지 검증 여부(디폴트 'Y'), 유효하지 않는 값 (디폴트 'N')

항목	타입	필수여부	설명
extract_feature_base64	Boolean	N	Liveness 검증을 완료한 이미지 하나에 대한 특징점 벡터값 추가 제공 유무 (byte 값을 base64로 제공)
fin_code	String	N	매칭에 대한 개별 임계치를 사용하는 경우에 설정한 임계치의 (백 오피스를 사용하는 경우 설정된 기관 코드, 미사용시 fin_code: "000", fin_name: "default")

- 요청 이미지들을 zip 포맷으로 압축하여, 압축 파일을 전송해야 합니다. 압축파일의 이름은 상관 없습니다.
- 요청 이미지들은 png / jpg 포맷을 지원하며, 이미지 파일의 이름은 1, 2, 3, ..., 30, 31, 32, ... 와 같이 1부터 frame_counts 개수까지의 순차적인 이름이어야 합니다.
- **이미지 이름의 숫자가 처리 순서**가 됩니다. 시간 순서가 있는 이미지들일 경우 파일명을 순서대로 설정하여 압축해주셔야 합니다.

일례로, 다음과 같은 내용을 포함한 zip 파일은 전송 가능합니다.

```
images1.zip (1.png, 2.png, 3.png, 4.png, 5.png, 6.png)
images2.zip (3.jpg, 4.jpg, 1.jpg, 2.jpg, 5.jpg)
```

그러나 다음과 같이 인식할 수 없는 이름이거나, 이미지 개수가 4장이 아닌 경우, 파일 확장자가 filename_extension과 다른 경우 전송이 불가능합니다.

```
images3.zip (1.png, img2.jpg, test3.jpg, 4.jpg)
images4.zip (1.jpg, 2.jpg, 3.jpg)
images5.zip (0.png, 1.png, img2.png, test3.png, 4.png)
```

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
200	실패	FAIL-0000	얼굴 검출 실패
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
400	오류	ERR-4008	'frame_counts' 필드가 존재하지 않음
400	오류	ERR-4009	'frame_counts' 필드의 값이 숫자 형식이 아님
400	오류	ERR-4010	'frame_counts' 필드의 값은 4 이상이어야만 함

Status Code	Type	return_code	Description
400	오류	ERR-4011	'filename_extension' 필드가 존재하지 않음
400	오류	ERR-4012	'frame_counts' 필드의 값이 실제 사진의 개수와 다름
400	오류	ERR-4013	'filename_extension' 필드의 확장자와 실제 사진의 파일 확장자가 다름
400	오류	ERR-4022	인식 엔진 - Multi frame liveness 요청의 zip 압축파일을 해제할 수 없음
400	오류	ERR-4023	인식 엔진 - Multi frame liveness 요청 압축 파일 내 이미지 획득할 수 없음
400	오류	ERR-4024	인식 엔진 - Multi frame liveness 요청 압축 파일 내 이미지 갯수가 다름
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	오류	ERR-4501	wrapper server form 전송 데이터 20MB 초과
400	오류	ERR-4503	암호화 된 정보를 해독하지 못했을 경우.
400	오류	ERR-4504	암호화에 실패했을 경우.
400	오류	ERR-4505	서버에서 전송받은 압축파일은 압축해제 실패시 에러
400	오류	ERR-4508	금융사의 config 사용이 활성화 되지 않은 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5009	인식 엔진 - 최대 얼굴 검출 개수 설정 에러
500	오류	ERR-5010	인식 엔진 - 얼굴 검출 최소/최대 사이즈 설정 에러
500	오류	ERR-5011	인식 엔진 - 얼굴 검출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5019	인식 엔진 - 얼굴 진위 여부 확인 중 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
----	----	----

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

신뢰도(confidence)는 0~1 사이의 실수 값이며, 1에 가까울수록 실제 얼굴(Live)입니다.

항목	타입	설명
confidence	Float	liveness 정도 (요청 및 응답 예시 참조)
is_live	Boolean	결과 값에 따른 심사 결과 (true: 자동승인, false: 자동거절)
threshold_info	JSON Object	결과에 따른 심사 기준 값
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)
compare_result	JSON Object	추가 매칭 결과 응답 코드와 메시지 (POST /compare 문서 참조)
feature_base64	JSON Object	추가 벡터값 추출 결과 Byte를 base64 encoding한 값

요청 및 응답 예시

```
curl -X POST http://127.0.0.1:{port}/liveness/multiframe/v2
--form 'images=@"multiframe.zip"' \
--form 'frame_counts="10"' \
--form 'filename_extension="png"'
{
  "confidence": 0.74912024,
  "is_live": false,
  "threshold_info": {
    "fin_code": "000",
    "fin_name": "default",
    "auto_approve": {
      "min": 0.8836,
      "max": 1.0
    },
    "auto_reject": {
      "min": 0.0,
      "max": 0.8835
    }
  },
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

```
curl -X POST http://127.0.0.1:{port}/liveness/multiframe/v2?
encrypt_mode=true
--form 'encrypted_images=@"/encrypted_multiframe.zip"' \
--form 'frame_counts="10"' \
--form 'filename_extension="png"'
{
  "confidence": 0.74912024,
  "is_live": false,
  "threshold_info": {
    "fin_code": "000",
    "fin_name": "default",
    "auto_approve": {
      "min": 0.8836,
      "max": 1.0
    },
    "auto_reject": {
      "min": 0.0,
      "max": 0.8835
    }
  },
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

compare / feature 기능 사용

```
curl -X POST http://127.0.0.1:{port}/liveness/multiframe/v2?
encrypt_mode=true
--form 'encrypted_images=@"/encrypted_multiframe.zip"' \
--form 'frame_counts="10"' \
--form 'filename_extension="png"' \
--form 'encrypted_compare_image=@"/encrypted_compare_image"' \
--form 'extract_feature_base64="true"'
{
  "confidence": 0.99462026,
  "is_live": true,
  "threshold_info": {
    "fin_code": "000",
    "fin_name": "default",
    "auto_approve": {
      "min": 0.7037,
      "max": 1.0
    },
    "auto_reject": {
      "min": 0.0,
      "max": 0.7036
    }
  }
}
```

```

    },
    "return_msg": {
      "return_code": "SUCC-0000",
      "return_msg": "OK."
    },
    "compare_result": {
      "similarity_confidence": 1.0,
      "match_result": 1,
      "threshold_info": {
        "fin_code": "000",
        "fin_name": "default",
        "auto_approve": {
          "min": 0.9998,
          "max": 1.0
        },
        "manual_review": {
          "min": 0.9985,
          "max": 0.9997
        },
        "auto_reject": {
          "min": 0.0,
          "max": 0.9984
        }
      },
      "return_msg": {
        "return_code": "SUCC-0000",
        "return_msg": "OK."
      }
    },
    "feature_base64": {
      "value": "h2UVPchD67zSGWY8MTTo4verz5Ty17CY9V1wDPQB...",
      "return_msg": {
        "return_code": "SUCC-0000",
        "return_msg": "OK."
      }
    }
  }
}

```

- ERR-4501

```

curl -X POST http://127.0.0.1:{port}/alignedfaces/v2?encrypt_mode=false --
form 'image=@image.jpg'
{
  "return_msg": {
    "return_code": "ERR-4501",
    "return_msg": "Maximum upload size exceeded (maximum: 20MB)"
  }
}

```

4.24. POST /liveness/multiframe/v2/unzip

API 개요

순차적으로 찍은 4장 이상의 이미지를 보내서 얼굴 진위 여부(Multi frame Liveness)를 확인합니다. 반드시 4장 이상의 동일 인물의 이미지 및 동일한 파일 확장자의 이미지가 필요합니다.

Request Query

Request Header

항목	내용	비고
Content-type	multipart/form-data	

Request Body

항목	타입	필수여부	설명
frame_counts	String	Y	Liveness를 측정할 이미지 파일의 개수
filename_extension	String	Y	Liveness를 측정할 이미지 파일의 확장자 이름
encrypt_mode	Boolean	N	암호화 모드 사용 유무 (디폴트 false)
image_01	blob	N	Liveness를 측정할 첫번째 이미지를 파일
...	...	...	...
image_30	blob	N	Liveness를 측정할 마지막(최대 개수 30) 이미지 파일

- 요청 이미지들은 png / jpg 포맷을 지원하며, 이미지 파일의 이름은 1, 2, 3, ..., 30, 31, 32, ... 와 같이 1부터 frame_counts 개수까지의 순차적인 이름이어야 합니다.
- 이미지 이름의 숫자가 처리 순서가 됩니다. 시간 순서가 있는 이미지들일 경우 파일명을 순서대로 설정하여야 합니다.

일례로, 다음과 같은 내용을 포함한 파일은 전송 가능합니다.

```
image_01: 1.png
image_02: 2.png
...
image_05: 5.png

---

image_01: 1.jpg
image_02: 2.jpg
...
image_05: 5.jpg
```

그러나 다음과 같이 인식할 수 없는 이름이거나, 이미지 개수가 4장이 아닌 경우, 파일 확장자가 filename_extension과 다른 경우 전송이 불가능합니다.

```
image_01: 1.png
image_02: 2.jpg
...
image_05: 5.jpg

---

image_01: 1.jpg
image_02: 2.png
...
image_05: 5.png

---

image_01: 1.jpg
image_02: 2.jpg
...
image_05: 5.jpg
```

Response Status Code

Status Code	Type	return_code	Description
200	성공	SUCC-0000	작업 성공
200	실패	FAIL-0000	얼굴 검출 실패
400	오류	ERR-4003	업로드된 이미지 파일 없음
400	오류	ERR-4004	이미지의 내용이 없거나 깨짐, 또는 잘못되어 디코딩 실패
400	오류	ERR-4008	'frame_counts' 필드가 존재하지 않음
400	오류	ERR-4009	'frame_counts' 필드의 값이 숫자 형식이 아님
400	오류	ERR-4010	'frame_counts' 필드의 값은 4 이상이어야만 함
400	오류	ERR-4011	'filename_extension' 필드가 존재하지 않음
400	오류	ERR-4012	'frame_counts' 필드의 값이 실제 사진의 개수와 다름
400	오류	ERR-4013	'filename_extension' 필드의 확장자와 실제 사진의 파일 확장자가 다름
400	오류	ERR-4022	인식 엔진 - Multi frame liveness 요청의 zip 압축파일을 해제할 수 없음
400	오류	ERR-4023	인식 엔진 - Multi frame liveness 요청 압축 파일 내 이미지 획득할 수 없음
400	오류	ERR-4024	인식 엔진 - Multi frame liveness 요청 압축 파일 내 이미지 갯수가 다름
406	오류	ERR-4025	해당 API의 사용이 제한됨
406	오류	ERR-4026	API 서버의 사용 기한이 만료됨

Status Code	Type	return_code	Description
429	오류	ERR-4027	API 서버로 일정시간 동안 너무 많은 요청이 들어와서 처리가 힘든 경우
400	오류	ERR-4501	wrapper server form 전송 데이터 20MB 초과
400	오류	ERR-4503	암호화 된 정보를 해독하지 못했을 경우.
400	오류	ERR-4504	암호화에 실패했을 경우.
400	오류	ERR-4506	전송받은 이미지 의 byte정보를 추출하지 못했을 경우.
500	오류	ERR-5006	얼굴인식엔진 초기화 에러
500	오류	ERR-5007	얼굴인식엔진 특징점 추출 모듈 초기화 에러
500	오류	ERR-5008	인퍼런스 전 얼굴인식엔진이 준비되지 않음.
500	오류	ERR-5009	인식 엔진 - 최대 얼굴 검출 개수 설정 에러
500	오류	ERR-5010	인식 엔진 - 얼굴 검출 최소/최대 사이즈 설정 에러
500	오류	ERR-5011	인식 엔진 - 얼굴 검출 중 에러
500	오류	ERR-5015	인식 엔진 초기화 해제 에러
500	오류	ERR-5019	인식 엔진 - 얼굴 진위 여부 확인 중 에러
500	오류	ERR-5022	얼굴인식엔진 특성 추출 모듈 초기화 에러
500	오류	ERR-5023	얼굴인식엔진 진위 여부 확인 모듈 초기화 에러
500	오류	ERR-5099	서버에서 일시적으로 요청을 정상적으로 처리하기 힘든 상태의 에러

Response Header

항목	내용	비고
Content-type	application/json	응답 코드와 실행 결과가 같이 전달됨 실행 실패시 응답 코드만 전달됨 (API 응답 예시 참조)

Response Body

신뢰도(confidence)는 0~1 사이의 실수 값이며, 1에 가까울수록 실제 얼굴(Live)입니다.

항목	타입	설명
confidence	Float	liveness 정도 (요청 및 응답 예시 참조)
is_live	Boolean	결과 값에 따른 심사 결과 (true: 자동승인, false: 자동거절)
threshold_info	JSON Object	결과에 따른 심사 기준 값

항목	타입	설명
return_msg	JSON Object	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

```
curl -X POST http://127.0.0.1:{port}/liveness/multiframe/v2/unzip
--form 'image_01=@"/1.jpg"' \
--form 'image_02=@"/2.jpg"' \
--form 'image_03=@"/3.jpg"' \
--form 'image_04=@"/4.jpg"' \
--form 'frame_counts="10"' \
--form 'filename_extension="png"'
{
  "confidence": 0.74912024,
  "is_live": false,
  "threshold_info": {
    "fin_code": "000",
    "fin_name": "default",
    "auto_approve": {
      "min": 0.8836,
      "max": 1.0
    },
    "auto_reject": {
      "min": 0.0,
      "max": 0.8835
    }
  },
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

```
curl -X POST http://127.0.0.1:{port}/liveness/multiframe/v2/unzip?
encrypt_mode=true
--form 'image_01=@"/1.jpg"' \
--form 'image_02=@"/2.jpg"' \
--form 'image_03=@"/3.jpg"' \
--form 'image_04=@"/4.jpg"' \
--form 'frame_counts="10"' \
--form 'filename_extension="png"'
{
  "confidence": 0.74912024,
  "is_live": false,
  "threshold_info": {
    "auto_approve": {
      "min": 0.8836,
      "max": 1.0
    },
  },
}
```



```
        "auto_reject": {
            "min": 0.0,
            "max": 0.8835
        },
        "return_msg": {
            "return_code": "SUCC-0000",
            "return_msg": "OK."
        }
    }
}
```

- ERR-4501

```
curl -X POST http://127.0.0.1:{port}/alignedfaces/v2?encrypt_mode=false --
form 'image=@image.jpg'
{
    "return_msg": {
        "return_code": "ERR-4501",
        "return_msg": "Maximum upload size exceeded (maximum: 20MB)"
    }
}
```

4.25. GET /threshold

API 개요

전체 민감도에 대한 기준값을 조회합니다. (compare, liveness)

Request Query

Request Header

Request Body

Response Status Code

Status Code	발생 조건	설명
200	성공	
500	실패	민감도 기준값 조회 실패

Response Header

항목	내용	비고
Content-type	application/json	API 응답 예시 참조

Response Body

항목	타입	설명
id	Integer	Identity
match_auto_approve	Integer	compare 자동승인 민감도 기준
match_manual_review	Integer	compare 수동심사 민감도 기준
liveness_auto_approve	Integer	liveness 자동승인 민감도 기준

요청 및 응답 예시

```
curl -X GET http://127.0.0.1:{port}/threshold
{
  "id": 2,
  "match_auto_approve": 5,
  "match_manual_review": 3,
  "liveness_auto_approve": 5
}
```

4.26. GET /threshold/compare

API 개요

compare 민감도에 대한 기준값을 조회합니다.

Request Query

Request Header

Request Body

Response Status Code

Status Code	발생 조건	설명
200	성공	
500	실패	compare 민감도 조회 실패

Response Header

항목	내용	비고
Content-type	application/json	API 응답 예시 참조

Response Body

항목	타입	설명
----	----	----

항목	타입	설명
match_auto_approve	Integer	compare 자동승인 민감도 기준
match_manual_review	Integer	compare 수동심사 민감도 기준

요청 및 응답 예시

```
curl -X GET http://127.0.0.1:{port}/threshold/compare
{
  "match_auto_approve": 5,
  "match_manual_review": 3,
}
```

4.27. PUT /threshold/compare

API 개요

compare 민감도에 대한 기준값을 수정합니다. 1 ~ 16 level까지 설정 가능합니다.

모델 별 디폴트 값

모델명	match_auto_approve	match_manual_review
FEATURE-L	13	12

Request Query

Request Header

항목	내용	비고
Content-type	application/json	

Request Body

항목	타입	필수 여부	설명
match_auto_approve	Integer	Y	compare 자동승인 민감도 기준 설정 값 (match_manual_review 값 보다 크거나 같아야 합니다)
match_manual_review	Integer	Y	compare 수동심사 민감도 기준 설정 값 (match_auto_approve 값 보다 작거나 같아야 합니다)

Response Status Code

Status Code	발생 조건	설명
-------------	-------	----

Status Code	발생 조건	설명
200	성공	
500	실패	compare 민감도 수정 실패

Response Header

항목	내용	비고
Content-type	application/json	API 응답 예시 참조

Response Body

항목	타입	설명
match_auto_approve	Integer	compare 자동승인 민감도 기준
match_manual_review	Integer	compare 수동심사 민감도 기준

요청 및 응답 예시

```
curl -X PUT http://127.0.0.1:{port}/threshold/compare
-H 'accept: application/json'
-H 'Content-Type: application/json'
-d '{
  "match_auto_approve": 5,
  "match_manual_review": 3
}'
```

4.28. GET /threshold/liveness

API 개요

liveness 민감도에 대한 기준값을 조회합니다.

Request Query

Request Header

Request Body

Response Status Code

Status Code	발생 조건	설명
200	성공	
500	실패	liveness 민감도 조회 실패

Response Header

항목	내용	비고
Content-type	application/json	API 응답 예시 참조

Response Body

항목	타입	설명
liveness_auto_approve	Integer	liveness 자동승인 민감도 기준

요청 및 응답 예시

```
curl -X GET http://127.0.0.1:{port}/threshold/liveness
{
  "liveness_auto_approve": 5
}
```

4.29. PUT /threshold/liveness

API 개요

liveness 민감도에 대한 기준값을 수정합니다. 1 ~ 7 level까지 설정 가능합니다.

모델 별 디폴트 값

모델명	liveness_auto_approve
LIVENESS-11	4
LIVENESS-12	6
LIVENESS-12-2	4
LIVENESS-13	4
LIVENESS-14	4
LIVENESS-14-3	4

Request Query

Request Header

항목	내용	비고
Content-type	application/json	

Request Body

항목	타입	필수여부	설명
liveness_auto_approve	Integer	Y	liveness 자동승인 민감도 기준 설정 값

Response Status Code

Status Code	발생 조건	설명
200	성공	
500	실패	liveness 민감도 수정 실패

Response Header

항목	내용	비고
Content-type	application/json	API 응답 예시 참조

Response Body

항목	타입	설명
liveness_auto_approve	Integer	liveness 자동승인 민감도 기준

요청 및 응답 예시

```
curl -X PUT http://127.0.0.1:{port}/threshold/liveness
-H 'accept: application/json'
-H 'Content-Type: application/json'
-d '{
  "liveness_auto_approve": 5
}'
```

4.30. GET /version-wrap

API 개요

face server java wrap 서버 버전 정보를 확인합니다.

Request Query

Request Header

Request Body

Response Status Code

Status Code	Type	return_code	Description
-------------	------	-------------	-------------

Status Code	Type	return_code	Description
200	성공	SUCC-0000	
500	오류		

Response Header

항목	내용	비고
Content-type	application/json	API 응답 예시 참조

Response Body

항목	타입	설명
wrap_version	String	java wrap 서버 버전 정보
return_msg	String	결과 응답 코드와 메시지 (3.2 응답 코드 목록 참조)

요청 및 응답 예시

```
curl -X GET http://127.0.0.1:{port}/version-wrap
{
  "wrap_version": "1.5.6-20240122.3",
  "return_msg": {
    "return_code": "SUCC-0000",
    "return_msg": "OK."
  }
}
```

4.31. GET /model/info

API 개요

요청한 sdk 버전에 맞는 model 정보를 반환 합니다.

Request Query

항목	설명	필수여부	예시
sdk	sdk 버전	Y	/model/info?sdk=1.16.0
liveness	liveness 모델 정보 포함 여부	Y	/model/info?liveness=true

Response Body

정상 동작시

항목	타입	설명
liveness	JSON Object	버전정보 및 threshold 정보
best_shot	JSON Object	버전정보 및 threshold 정보

오류 발생시 return_msg JSON 객체만 반환됩니다.

항목	타입	설명
liveness	JSON Object	항상 null (무시 가능)
best_shot	JSON Object	항상 null (무시 가능)
result	JSON Object	결과 응답 메세지

요청 및 응답 예시

```
curl {host}/model/info?sdk=1.16.0&liveness=true
```

- 동작 성공시

```
{
  "result": {
    "message": "OK"
  },
  "content": {
    "liveness": {
      "version": "LIVENESS-14",
      "threshold": {
        "n": 0.9684
      }
    },
    "best_shot": {
      "version": "FEATURE-L",
      "threshold": {
        "aq": 0.5648,
        "q": 63.96,
        "l": 0.9,
        "m": 0.5
      }
    }
  }
}
```

4.32. GET /model/aos

API 개요

AOS 용 전체 모델을 다운로드 합니다.

Request Query

항목	설명	필수여부	예시
sdk	sdk 버전	Y	/model/aos?sdk=1.16.0
alg_type	모델 암호화 타입	Y	/model/aos?alg_type=seed

※ 현재 AOS 용 model 의 alg_type 은 seed 만 사용 가능합니다.

Response Body

정상 동작시 model zip 파일이 Byte Array 형태로 반환됩니다.

항목	타입	설명
-	Byte Array	model zip 파일 (Byte Array)

오류 발생시 return_msg JSON 객체만 반환됩니다.

항목	타입	설명
result	JSON Object	결과 응답 메세지

요청 및 응답 예시

```
curl {host}/model/aos?sdk=1.16.0&alg_type=aes
```

- 동작 성공시
Byte String이 응답으로 전달됩니다. 다음의 문자열은 Byte String임을 표현하기 위한 것으로, 정상입니다.

```
E   R   ; ep<[G ... (Byte string)
```

- 오류 발생시

```
{
  "content": null,
  "result": {
    "message": "can not load model file."
  }
}
```

4.33. GET /model/ios

IOS 용 전체 모델을 다운로드 합니다.

항목	설명	필수여부	예시
sdk	sdk 버전	Y	/model/ios?sdk=1.16.0
alg_type	모델 암호화 타입	Y	/model/ios?alg_type=seed

정상 동작시 model zip 파일이 Byte Array 형태로 반환됩니다.

오류 발생시 return_msg JSON 객체만 반환됩니다.

요청 및 응답 예시

- 동작 성공시

Byte String이 응답으로 전달됩니다. 다음의 문자열은 Byte String임을 표현하기 위한 것으로, 정상입니다.

- 오류 발생시

4.34. GET /model/aos/best-shot

API 개요

AOS 용 best-shot 모델을 다운로드 합니다.

Request Query

항목	설명	필수여부	예시
sdk	sdk 버전	Y	/model/aos/best-shot?sdk=1.16.0
alg_type	모델 암호화 타입	Y	/model/aos/best-shot?alg_type=seed

※ 현재 AOS 용 model 의 alg_type 은 seed 만 사용 가능합니다.

Response Body

정상 동작시 model zip 파일이 Byte Array 형태로 반환됩니다.

항목	타입	설명
-	Byte Array	model zip 파일 (Byte Array)

오류 발생시 return_msg JSON 객체만 반환됩니다.

항목	타입	설명
result	JSON Object	결과 응답 메세지

요청 및 응답 예시

```
curl {host}/model/aos/best-shot?sdk=1.16.0&alg_type=aes
```

- 동작 성공시
Byte String이 응답으로 전달됩니다. 다음의 문자열은 Byte String임을 표현하기 위한 것으로, 정상입니다.

```
E          ; ep<[G ... (Byte string)
```

- 오류 발생시

```
{
  "content": null,
  "result": {
    "message": "can not load model file."
  }
}
```

4.35. GET /model/aos/liveness

API 개요

AOS 용 liveness 모델을 다운로드 합니다.

Request Query

항목	설명	필수여부	예시
sdk	sdk 버전	Y	/model/aos/liveness?sdk=1.16.0
alg_type	모델 암호화 타입	Y	/model/aos/liveness?alg_type=seed

※ 현재 AOS 용 model 의 alg_type 은 seed 만 사용 가능합니다.

Response Body

정상 동작시 model zip 파일이 Byte Array 형태로 반환됩니다.

항목	타입	설명
-	Byte Array	model zip 파일 (Byte Array)

오류 발생시 return_msg JSON 객체만 반환됩니다.

항목	타입	설명
result	JSON Object	결과 응답 메세지

요청 및 응답 예시

```
curl {host}/model/aos/liveness?sdk=1.16.0&alg_type=aes
```

- 동작 성공시
Byte String이 응답으로 전달됩니다. 다음의 문자열은 Byte String임을 표현하기 위한 것으로, 정상입니다.

```
E0@00R000;0ep<[G0... (Byte string)
```

- 오류 발생시

```
{
  "content": null,
  "result": {
    "message": "can not load model file."
  }
}
```

4.36. GET /model/ios/best-shot

IOS 용 best-shot 모델을 다운로드 합니다.

항목	설명	필수여부	예시
sdk	sdk 버전	Y	/model/ios/best-shot?sdk=1.16.0
alg_type	모델 암호화 타입	Y	/model/ios/best-shot?alg_type=seed

정상 동작시 model zip 파일이 Byte Array 형태로 반환됩니다.

항목	타입	설명
-	Byte Array	model zip 파일 (Byte Array)

오류 발생시 return_msg JSON 객체만 반환됩니다.

항목	타입	설명
result	JSON Object	결과 응답 메시지

```
curl {host}/model/ios/best-shot?sdk=1.16.0&alg_type=aes
```

- Byte String이 응답으로 전달됩니다. 다음의 문자열은 Byte String임을 표현하기 위한 것으로, 정상입니다.

```
E@R;?ep<[G... (Byte string)
```

- ```
{
 "content": null,
 "result": {
```

```
 "message": "can not load model file."
 }
}
```

4.37. GET /model/ios/liveness

API 개요

IOS 용 liveness 모델을 다운로드 합니다.

Request Query

| 항목       | 설명        | 필수여부 | 예시                                |
|----------|-----------|------|-----------------------------------|
| sdk      | sdk 버전    | Y    | /model/ios/liveness?sdk=1.16.0    |
| alg_type | 모델 암호화 타입 | Y    | /model/ios/liveness?alg_type=seed |

Response Body

정상 동작시 model zip 파일이 Byte Array 형태로 반환됩니다.

| 항목 | 타입         | 설명                        |
|----|------------|---------------------------|
| -  | Byte Array | model zip 파일 (Byte Array) |

오류 발생시 return\_msg JSON 객체만 반환됩니다.

| 항목     | 타입          | 설명        |
|--------|-------------|-----------|
| result | JSON Object | 결과 응답 메시지 |

요청 및 응답 예시

```
curl {host}/model/ios/liveness?sdk=1.16.0&alg_type=aes
```

- 동작 성공시  
Byte String이 응답으로 전달됩니다. 다음의 문자열은 Byte String임을 표현하기 위한 것으로, 정상입니다.

```
E     R   ; ep<[G ... (Byte string)
```

- 오류 발생시

```
{
 "content": null,
```

```
 "result": {
 "message": "can not load model file."
 }
 }
}
```

4.38. GET /model/aos/date

API 개요

AOS 용 전체 모델의 수정날짜를 반환합니다.

Request Query

| 항목       | 설명        | 필수여부 | 예시                            |
|----------|-----------|------|-------------------------------|
| sdk      | sdk 버전    | Y    | /model/aos/date?sdk=1.16.0    |
| alg_type | 모델 암호화 타입 | Y    | /model/aos/date?alg_type=seed |

Response Body

정상 동작시

| 항목         | 타입       | 설명          |
|------------|----------|-------------|
| updated_at | ISO 8601 | 모델파일의 수정 날짜 |

오류 발생시 return\_msg JSON 객체만 반환됩니다.

| 항목         | 타입          | 설명              |
|------------|-------------|-----------------|
| updated_at | JSON Object | 항상 null (무시 가능) |
| result     | JSON Object | 결과 응답 메세지       |

요청 및 응답 예시

```
curl {host}/model/aos/date?sdk=1.16.0&alg_type=seed
```

- 동작 성공시

```
{
 "result": {
 "message": "OK"
 },
 "content": {
 "updated_at": "2024-06-19T11:48:36.621547"
 }
}
```

```
 }
 }
}
```

4.39. GET /model/ios/date

API 개요

IOS 용 전체 모델의 수정날짜를 반환합니다.

Request Query

| 항목       | 설명        | 필수여부 | 예시                            |
|----------|-----------|------|-------------------------------|
| sdk      | sdk 버전    | Y    | /model/ios/date?sdk=1.16.0    |
| alg_type | 모델 암호화 타입 | Y    | /model/ios/date?alg_type=seed |

Response Body

정상 동작시

| 항목         | 타입       | 설명          |
|------------|----------|-------------|
| updated_at | ISO 8601 | 모델파일의 수정 날짜 |

오류 발생시 return\_msg JSON 객체만 반환됩니다.

| 항목         | 타입          | 설명              |
|------------|-------------|-----------------|
| updated_at | JSON Object | 항상 null (무시 가능) |
| result     | JSON Object | 결과 응답 메시지       |

요청 및 응답 예시

```
curl {host}/model/ios/date?sdk=1.16.0&alg_type=seed
```

- 동작 성공시

```
{
 "result": {
 "message": "OK"
 },
 "content": {
 "updated_at": "2024-06-19T11:48:36.621547"
 }
}
```



4.40. GET /model/aos/best-shot/date

API 개요

AOS 용 best-shot 모델의 수정날짜를 반환합니다.

Request Query

| 항목       | 설명        | 필수여부 | 예시                                      |
|----------|-----------|------|-----------------------------------------|
| sdk      | sdk 버전    | Y    | /model/aos/best-shot/date?sdk=1.16.0    |
| alg_type | 모델 암호화 타입 | Y    | /model/aos/best-shot/date?alg_type=seed |

Response Body

정상 동작시

| 항목         | 타입       | 설명          |
|------------|----------|-------------|
| updated_at | ISO 8601 | 모델파일의 수정 날짜 |

오류 발생시 return\_msg JSON 객체만 반환됩니다.

| 항목         | 타입          | 설명              |
|------------|-------------|-----------------|
| updated_at | JSON Object | 항상 null (무시 가능) |
| result     | JSON Object | 결과 응답 메시지       |

요청 및 응답 예시

```
curl {host}/model/aos/best-shot/date?sdk=1.16.0&alg_type=seed
```

- 동작 성공시

```
{
 "result": {
 "message": "OK"
 },
 "content": {
 "updated_at": "2024-06-19T11:48:36.621547"
 }
}
```

4.41. GET /model/aos/liveness/date

API 개요

AOS 용 liveness 모델의 수정날짜를 반환합니다.

Request Query

| 항목       | 설명        | 필수여부 | 예시                                     |
|----------|-----------|------|----------------------------------------|
| sdk      | sdk 버전    | Y    | /model/aos/liveness/date?sdk=1.16.0    |
| alg_type | 모델 암호화 타입 | Y    | /model/aos/liveness/date?alg_type=seed |

Response Body

정상 동작시

| 항목         | 타입       | 설명          |
|------------|----------|-------------|
| updated_at | ISO 8601 | 모델파일의 수정 날짜 |

오류 발생시 return\_msg JSON 객체만 반환됩니다.

| 항목         | 타입          | 설명              |
|------------|-------------|-----------------|
| updated_at | JSON Object | 항상 null (무시 가능) |
| result     | JSON Object | 결과 응답 메시지       |

요청 및 응답 예시

```
curl {host}/model/aos/liveness/date?sdk=1.16.0&alg_type=seed
```

- 동작 성공시

```
{
 "result": {
 "message": "OK"
 },
 "content": {
 "updated_at": "2024-06-19T11:48:36.621547"
 }
}
```

4.42. GET /model/ios/best-shot/date

API 개요

IOS 용 best-shot 모델의 수정날짜를 반환합니다.

Request Query

| 항목       | 설명        | 필수여부 | 예시                                      |
|----------|-----------|------|-----------------------------------------|
| sdk      | sdk 버전    | Y    | /model/ios/best-shot/date?sdk=1.16.0    |
| alg_type | 모델 암호화 타입 | Y    | /model/ios/best-shot/date?alg_type=seed |

Response Body

정상 동작시

| 항목         | 타입       | 설명          |
|------------|----------|-------------|
| updated_at | ISO 8601 | 모델파일의 수정 날짜 |

오류 발생시 return\_msg JSON 객체만 반환됩니다.

| 항목         | 타입          | 설명              |
|------------|-------------|-----------------|
| updated_at | JSON Object | 항상 null (무시 가능) |
| result     | JSON Object | 결과 응답 메시지       |

요청 및 응답 예시

```
curl {host}/model/ios/best-shot/date?sdk=1.16.0&alg_type=seed
```

- 동작 성공시

```
{
 "result": {
 "message": "OK"
 },
 "content": {
 "updated_at": "2024-06-19T11:48:36.621547"
 }
}
```

4.43. GET /model/ios/liveness/date

API 개요

IOS 용 liveness 모델의 수정날짜를 반환합니다.

Request Query

| 항목  | 설명     | 필수여부 | 예시                                  |
|-----|--------|------|-------------------------------------|
| sdk | sdk 버전 | Y    | /model/ios/liveness/date?sdk=1.16.0 |

| 항목       | 설명        | 필수여부 | 예시                                     |
|----------|-----------|------|----------------------------------------|
| alg_type | 모델 암호화 타입 | Y    | /model/ios/liveness/date?alg_type=seed |

Response Body

정상 동작시

| 항목         | 타입       | 설명          |
|------------|----------|-------------|
| updated_at | ISO 8601 | 모델파일의 수정 날짜 |

오류 발생시 return\_msg JSON 객체만 반환됩니다.

| 항목         | 타입          | 설명              |
|------------|-------------|-----------------|
| updated_at | JSON Object | 항상 null (무시 가능) |
| result     | JSON Object | 결과 응답 메시지       |

요청 및 응답 예시

```
curl {host}/model/ios/liveness/date?sdk=1.16.0&alg_type=seed
```

- 동작 성공시

```
{
 "result": {
 "message": "OK"
 },
 "content": {
 "updated_at": "2024-06-19T11:48:36.621547"
 }
}
```